



C Companion

Soradech Krootjohn

Table of Contents

Chapter	Topic	Page
1	Introduction to Computer Language and Flowchart	1
	Programming Language.....	1
	Types of Programming Language.....	2
	History of C Language.....	3
	Compilation Process in C Language.....	4
	Flowchart Diagram.....	5
	Exercise.....	9
2	Introduction to C Programming	10
	Header File, Preprocessing, and #include Directive.....	10
	Defining Function.....	11
	Printing String to Computer Screen.....	12
	Editing, Compiling, Executing, and Debugging.....	13
	Identifier.....	14
	Variable and Assignment.....	14
	Statement.....	16
	Compound Statement.....	16
	Expression.....	17
	Operator.....	18
	Precedence of Arithmetic Operators.....	20
	Named Constant and Preprocessor Macro.....	21
	Exercise.....	22
3	Input and output	23
	Output with <i>printf()</i>	23
	Field Width, Justification, and Precision.....	25
	Input with <i>scanf()</i>	26

	More on Output Stream Functions.....	28
	More on Input Stream Functions.....	29
	Exercise.....	31
4	Control Statement	32
	Selection Statement.....	32
	The <i>if</i> Statement.....	33
	The <i>if-else</i> Statement.....	35
	Nested <i>if</i> and <i>if-else</i> Statement.....	36
	The <i>switch-case</i> and <i>break</i> Statement.....	37
	Conditional Operator.....	41
	Exercise.....	42
5	Iteration	43
	The <i>while</i> Statement.....	43
	The <i>do-while</i> Statement.....	44
	The <i>for</i> Statement.....	46
	Compound Assignment Operator.....	47
	Increment and Decrement Operators.....	48
	Exercise.....	49
6	Array Variable	50
	One-Dimensional Array.....	50
	Array and <i>for</i> Statement.....	53
	Exercise.....	55
7	Pointer and String	56
	Indirection Operator	56
	Using <i>Address-Of</i> Operator.....	58
	Using <i>const</i> in Pointer Declaration.....	59
	Pointer and Array.....	60
	String.....	62
	Exercise.....	65

8	Structure Variable	67
	Declaring Structure Variable.....	68
	Accessing Structure's Elements.....	68
	Array of Structure Data.....	70
	<i>typedef</i>	71
	Using <i>typedef</i> for Structure.....	72
	Exercise.....	74
9	Function	75
	Types of Function.....	76
	Function with no parameter and no return.....	77
	Function with parameter but has no return.....	78
	Function with no parameter but has return.....	79
	Function with parameter and return.....	80
	Pass by value versus pass by reference.....	81
	Passing an array to a function.....	82
	Passing a structure variable to a function.....	83
	Exercise.....	84
10	Command Line Argument	85
	Using Command Line Argument in a Program.....	86
	Problem with Using Command Line Argument.....	88
	Exercise.....	90
11	Stream and File	91
	What is a file?.....	91
	Types of file.....	91
	File Stream.....	92
	Open a Stream to File.....	92
	Error Handling for <i>fopen()</i>	94
	Close the File Stream.....	95
	Reading Data from a Text File.....	96

	Reading a Line of Characters from a File.....	97
	Writing Data to a Text File.....	98
	Writing One Line at a Time.....	99
	Writing a Block of Data to a File.....	100
	Reading a Block of Data.....	101
	Manipulating Direct Access.....	102
	Exercise.....	103
12	Project File	104
	Exercise.....	107
	Bibliography	108

Chapter 1

Introduction to Computer Language and Flowchart

Learning outcome

After completing this chapter, students should be able to:

1. Tell what the characteristics of low-level language are
2. Tell what the characteristics of high-level language are
3. Provide example of low-level language
4. Provide example of high-level language
5. Explain how C compilation functions
6. Draw flowchart to describe how thing works

Programming Language

A programming language is a set of instructions following a particular grammatical rules for instructing computer devices to perform specific tasks. It is written by people who are called programmers. Programming Languages are mainly classified as low-level and high-level Languages. Low-level language is also called “machine language” since it is more understandable by machine rather than human. Low-level language consists of byte of instructors and data. Assembly language is a good example of such a language. High-level language, on the other hand, is more understandable by humans. It was designed to be close to human language, say English. As a result, it is a lot easier to learn and comprehend. Some well-known high-level languages include Basic, C, Fortran, Pascal, Java, etc.

At present, most software programmers use high-level languages to create application programs because they are easy to write and can be written quicker since most projects become more complex and larger scale. However, there are so many languages to choose from. It is hard to tell which one is better than the others. One language may be suitable for one particular task, while others may perform better in some other task. The choice of language has been an issue to discuss with. Some programmers may prefer to stick with a language they have been using for a long time. But they may have to switch to use other languages, which can work better on their undergoing job than the language they are familiar with. Others may consider using a language based on its available development tools. There is no conclusion on which language is the best fit on all tasks. In this book, however, C programming

language is considered one of the most popular languages used by programmers. It is a core language used in various microcontrollers and very good at creating a variety of types of applications. Most people start their first programming skill with C language and then extend to other languages such as C++, C#, Java, etc., and other web programming languages such as PHP, Javascript and so on. And here are some good reasons why C language is preferred by most software developers.

- C language is a very powerful and flexible language. It can be used to create projects including operating systems, word processors, graphics, spreadsheets, and even compilers for other languages.
- There are so many useful libraries provided so that it makes the development process more efficient.
- C language was standardized by ANSI (American National Standard Institute), which makes the set of rules for C compilers. This helps C language become more portable, meaning that a C program can be written on one computer system and can be compiled and run on another system.
- C language is modular and structured programming. The codes can be written as functions, which can be called by other parts of a program as many times as wished. Additionally, the functions themselves can be reused in other projects over and over as well.

There are still some other good sides of C language. This is quite a good reason why C language should be used as our first programming language.

Types of Programming Language

The main goal of programming languages is to instruct the computer system to perform specific tasks according to the statements written in the program. There are two main types of programming languages: compiled language and interpreted language.

A compiled language relies on its compiler to translate a source program (source code) into machine-understandable code, called executable code, which is stored in a binary file. If a source program contains even one error, the translation will be aborted and the creation of an executable file is a failure. This means that a compiler requires that the source program has to be free of error so that the translation process becomes a success. Once the executable file has been created, it can be run only on a specific operating system it was compiled into. As a result, an executable file used on Microsoft Windows cannot be run on the Mac OS, nor any distributions of Linux. Nevertheless, after the successful creation of an executable file, the source program is no longer needed, unless the program is fixed or updated and has to be recompiled.

An Interpreted language, on the other hand, relies on its interpreter to translate the source code into machine-understandable instructions at runtime. This means that a source code written in an interpreted language can be run immediately after the code has been written. Each statement will be run one-by-one after it has been read by the interpreter. If an error is found, the interpretation process stops at the statement that causes the error. Since a program written in an interpreted language does not transform its source code into an executable file, the source program is always needed during the execution.

A clear difference between these two types is that the compiler performs its translation as a whole at once whereas the interpreter performs its translation one statement at a time, sometimes called line-by-line translation. Because a compiled program is transformed into an executable code, it runs on computer system faster than the interpreted program. Therefore, in applications that the execution speed is a key factor, the compiled programs are more preferable. However, the interpreted programs are quick to create and debug. They are often used to create mockup or prototype programs.

C language is considered as a compiled language because most C language vendors make only C compilers to support the development of C programs.

History of C Language

C language was first developed by Dennis Ritchie between 1969 and 1973 at AT&T Bell Labs. It was used to re-implement the Unix operating system. Ritchie called this newly developed language, C, simply because there was already a B programming language at his laboratory. C language is easier to understand and code. Later, it was used to replace the coding in Assembly language and has since become one of the most widely used programming languages of all time. In 1989, C language was standardized by ANSI, (ANSI C), and later by ISO (International Organization for Standardization). Various vendors distributed their own C compiler such as Turbo C, Borland C, CC, Microsoft CL, etc. At the advent of the Linux operating system and the open source community, C compiler has since been included in the Linux distributions. GNU C Compiler (gcc) is the main building tool used to create the Linux operating system itself as well as other applications running on the Linux system. The use of C Language is not limited only on desktop computers but it is also used in some other platforms ranging from supercomputers to embedded systems.

Compilation Process in C Language

The main purpose of the compilation process is to translate a source program into an executable file. But, in fact, this process can be broken down into 4 phases: 1) Pre-processing, 2) Compilation, 3) Assembly, and 4) Linking.

Pre-processing

In this phase, all comments in the source code are removed. All macros and included files are expanded. Given a source file, say `hello.c`, the preprocessor will produce the contents of the header file(s) joined with the contents of the `hello.c`. In this case, a file called `hello.i` may be created if needed.

Compilation

This may sound a little confusing by its name, compilation, but it is actually a part of the compilation process. In this phase, the preprocessed code (`hello.i`) is translated into assembly level instructions, an intermediate human readable language. Because of this phase, it is legal to add inline assembly instructions in a C coding program. The output file of this phase is `hello.s`. However, some compilers employ an integrated assembler, which skips the generation of the intermediate assembly instructions and directly creates machine code instead.

Assembly

In this phase, the intermediate assembly instruction (`hello.s`) is given as input and will be translated by assembler into machine level instructions. This phase generates an object file, `hello.o`, which actually contains the instructions to be run on the target processor.

Linking

This is the final phase at which the executable file is generated. In this phase, Linker performs some extra work by rearranging the object codes that are out of order or missing. Linker locates where the functions are implemented so that they can be called by other pieces of codes. It also adds some code which is required when the program starts and ends. Some added code is responsible for setting up the environment like passing command line arguments. If all works well, the target executable file is created and, finally, this is the end of the whole compilation process.

Flowchart Diagram

What is a flowchart?

A flowchart is an organized combination of shapes, lines, and text that graphically illustrates a process or structure. Flowchart plays an important part in learning programming because it employs graphical representation to simplify the steps of the programming process. This makes it easier for programming learners to understand program's operations and translate it into a coding program. Since flowchart is language-independent, it can be used in any languages of choice.

Use of Flowchart

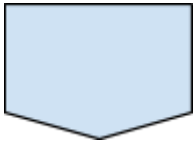
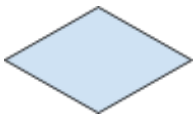
A flowchart can be used to define and analyze processes. For example, we can describe a registration process for new freshman students. The pictorial representation is more intuitive and understandable than that of the textual presentation. In addition, using a step-by-step picture of the process can shorten the time for discussing and communicating among team members.

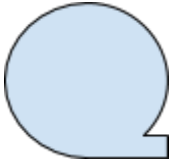
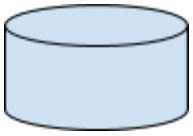
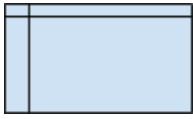
Flowcharts are recognized as one of the most important tools to reduce a gap between people with different knowledge backgrounds, educations, and positions. People may spend hours of talking, discussing, and coming up with a conclusion on what they are trying to complete. Flowchart is a universal language that can put everybody on the same page and make things done quicker. Once a flowchart is completed, it can be used for training, troubleshooting, testing, and inspecting processes.

Flowchart Symbol Definition

There are a variety of flowchart symbol definitions. They may be slightly different on some particular symbols. However, they share more common features than different ones. The most important thing to remember is when using these symbols, we have to be consistent with their meanings. The common flowchart symbols used in the programming process and their meaning are given in Table 1-1.

Table 1-1. Flowchart Symbol Description

Symbol	Name	Description
	Process	It represents any process, function, or action and is the most frequently used symbol in flowcharting.
	Document	It is used to represent any type of hard copy input or output, i.e. reports.
	Multi-document	It is the same as Document symbol except that it defines multiple documents
	Off-Page Connector	It is used to indicate that the flowchart continues on another page. Usually, the page number is placed in the shape for easy reference.
	Input/Output	It represents data that is available for input or resulting from processing, i.e. customer database records.
	Decision	It is used as a junction where a decision must be made. A single entry may have any number of alternative solutions, but only one can be chosen.
	Connector	It represents the exit to, or entry from, another part of the same flowchart. It is usually used to break a flow line that will be continued elsewhere.
	Terminator	It defines a start or stop point in a process.
	Manual Input	It represents the way of manually inputting data into a system.
	Display	It represents a machine display, i.e. computer monitor.
	Pre-defined Process	It represents a formally defined sub-process.

	Stored Data	It is generally a data storage flowchart symbol.
	Sequential Access Storage	It is an old reel of tape (magnetic tape), which nowadays is quite obsolete.
	Magnetic Disk	It simply refers to Database.
	Internal Storage	It represents data stores in memory.
	Direct Access Storage	It refers to storage on a hard drive.
	Flow Line	It indicates the direction of flow for material and/or information.

The following diagram illustrates a process of using an ATM to withdraw money from a saving account.

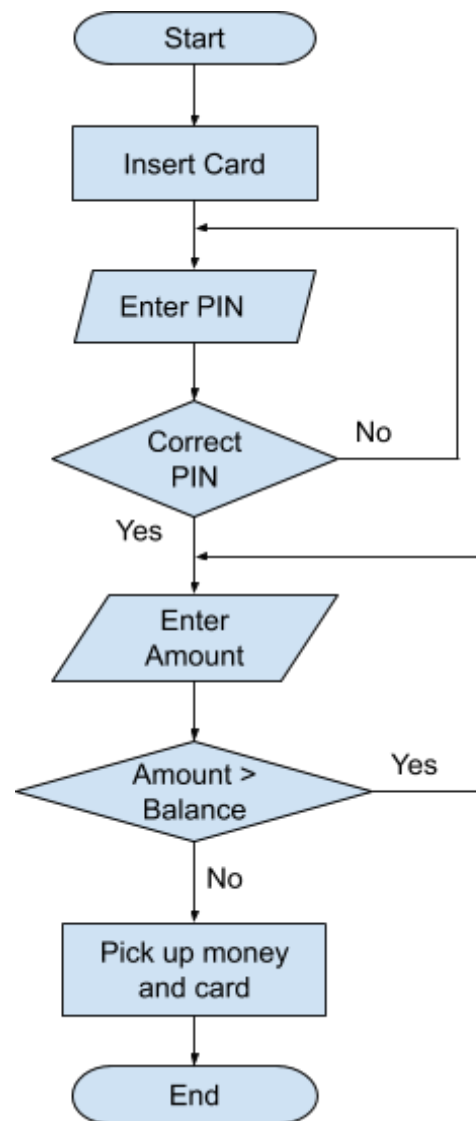


Figure 1-1. ATM withdrawal procedure

From the diagram in Figure 1-1, the process starts with card insertion by a user. Then, the user enters a four-digit PIN to the machine. If the PIN is incorrect, the user has to re-enter the numbers. If the right PIN is given, the user will be asked to enter the amount he/she wishes to withdraw. Only the amount that is less than or equal to the balance is allowed. Otherwise, the user is required to re-enter the amount. Upon successful, money is dispensed and the card is returned. User then takes the money and card. The transaction is completed.

There can be a lot of details going on behind this operation. Practically, a user has only 3 chances to enter the PIN. Exceeding the number of incorrect PIN trials will cause the card being held by the machine. User has to contact the bank to redeem his/her card. In other cases, most banks require their clients to maintain a minimum balance to continue the use of their account. Thus, withdrawal amount must follow this requirement. Nevertheless, this flowchart provides a simple ideal of how a withdrawal transaction on an ATM is performed. In reality, the process can be a little more complex than what is shown in this example.

Exercise

1. What are the characteristics of low-level language? Provide some examples of low-level language.
2. What are the characteristics of high-level language? Provide some examples of high-level language.
3. Describe the procedure of translating a source file into an executable file. Explain briefly on what each step does.
4. Draw a flowchart of how you watch a movie from a DVD player.

Chapter 2

Introduction to C Programming

Learning outcome

After completing this chapter, students should be able to:

1. Write a program to use directive macro to import a header file
2. Create an executable file using a C compiler
3. Tell how to declare a variable according to identifier naming rules
4. Write a program using arithmetic operators

Header File, Preprocessing, and *#include* Directive

C implementation provides *header files* containing the necessary declarations for various parts of the library. For example, the header file *math.h* contains declarations for mathematical functions, the header file *stdio.h* contains declarations for standard input and output functions.

The C compiler provides a simple way of effectively inserting header files into a source file, which can be controlled with *preprocessing directives*, all of which begin with the symbol #.

An *#include* directive designates a file whose contents are to be inserted into the program text. When the preprocessor encounters an *#include directive*, it replaces the directive by the contents of the designated file.

An example of how to use *stdio* library is shown below:

```
#include <stdio.h>
```

Defining Function

C programming is based on functions. The C function definition has the following form:

```
heading
{
    declarations (if any)
    statements
}
```

heading provides the *name* of the function, the *types* of arguments (if any), and *type* of the returned value. The *must-have* function that needs to run a C program is the main function, *main()*. An example of a C program is shown in Listing 2-1.

```
/* File: hello.c */
#include <stdio.h>

int main()
{
    printf("Hello world!\n");
    return 0;
}
```

Listing 2-1. hello.c

The heading *int main()* gives the name of the function as *main*. The empty parentheses means that the function takes no arguments. The *int* indicates that the function returns an integer value. The body of the function is a block containing two statements:

```
{
    printf("Hello world!\n");
    return 0;
}
```

Each statement ends with a semicolon. The value returned by *main()* is a *return code* that indicates whether the program runs successfully. Usually, a return code of 0 indicates a successful run.

Printing String to Computer Screen

The library function *printf()* is a powerful function for formatting and printing output. The functions in *stdio.h* access input and output devices via streams. Each stream is connected to a source of destination for data, such as a keyboard, a display, a printer, or a disk file. The function *printf()* writes data to the standard output stream, *stdout*, which is normally connected to the user's display monitor. Some examples of how to use *printf()* are given below:

```
printf("abc");  
printf("def");  
printf("ghi");
```

The result from executing the above statements are given below.

```
abcdefghi
```

Note: *printf()* does not automatically cause the new line after a string has been printed. If we wish the strings to be printed on separate lines, we must include new-line characters in the string literals as follows.

```
printf("abc\n");  
printf("def\n");  
printf("ghi\n");
```

These statements produce the output as

```
abc  
def  
ghi
```

An alternative, simpler, way to do so is to incorporate the new-line characters into a single statement as:

```
printf("abc\ndef\nghi\n");
```

Editing, Compiling, Linking, Executing, and Debugging

The text of a C program is stored in one or more source files, which can be created and revised with the aid of a text editor, such as *notepad* (in Windows), *vi* or *gedit* (in Linux), etc.

We use a *C compiler* to translate each source file into a machine-coded object file. Each source file is compiled separately.

The *linker* combines the object files with one another and with the code for any library functions called by the program. The output from the linker is an executable file, which contains all the machine code for the program and is ready to be run on the target computer.

Here, we use *gcc* to compile the *hello.c* on Linux:

```
$gcc -c hello.c
```

The option *-c* given above tells *gcc* to create an object file from the source file. We get *hello.o* as the *object file* from the above compilation. Then, we link the object file (in this case, we have just one file) to produce an executable file. The *-o* indicates the executable file to be produced named “*hello*”.

```
$gcc hello.o -o hello
```

We can now execute the program as follow:

```
$/hello
```

After executing, the result is given as shown below.

```
Hello World!
```

If we have only a single source file to be compiled, we may not need to generate the object file. This can simply be done by using:

```
$gcc hello.c -o hello
```

which produces the same result as the above operations. (Except that the object file will not be created.)

However, we can use a short command as *\$gcc hello.c*, which will produce a default executable file named *a.out*. We may then type *\$/a.out* to run it.

Note: If we encounter any problems during the compilation, we may need to edit (*debug*) the program. Even if the program may have passed the compilation, it may produce undesirable output. In this case, we might need some debugging tools, i.e. *gdb*, to correct the problems.

Identifier

In C programming, the identifiers (variables and functions) must be formed according to certain rules:

- An identifier can contain only letters, digits, and the underscore character, `_`.
- An identifier must begin with a letter of the alphabet or an underscore character. Thus `_func_call` is a valid identifier but `1st_round` is **not**.
- An identifier must not be the same as one of the *keywords* in C.
- C distinguishes between uppercase and lowercase letters, so that `amount`, `Amount`, and `AMOUNT` are three different identifiers.
- An identifier defined by programmer should not be the same as one already defined in the library. For example, `printf` should not be used as a programmer-defined identifier.

Variable and Assignment

A *variable* is a memory location named by an identifier. The value stored in the memory location is the current *value of the variable*. This value can be changed by an *assignment* operation, which stores a new value in the memory location. The following declarations

```
int count;
double amount;
```

declare `count` as integer variable and `amount` as double variable. We can declare several variables of the same type in a single declaration as:

```
int i, j, k;
double w, x, y, z;
```

We can give an initial value to a variable with an equal sign to the variable we want to declare.

```
int count = 100;
double amount = 29.95;
```

Likewise,

```
int i = 10, j = 20, k = 30;
```

is an example of multiple initializations to multiple variables.

Declarations in a block (a pair of braces {...}) should be given before the statements. For example, the block

```
{
    int count = 100;
    double amount = 29.95;

    count = 150;
    amount = 49.95;
}
```

declares *count* and *amount*, then assigns new value to each variable.

Variables declared within a block are called *local variables*. A local variable can be referred to only from within the block in which it is declared. Such variables cannot be used outside the block they are declared in.

Built-in data types provided by C language are listed in Table 2-1.

Table 2-1. Built-in data types in C language

Data Type	Size in byte	Description
char	1	A character or one-byte integer
short	2	Small integer number
int	4	Integer number
long	4	Integer number
long long	4	Big integer number
float	4	Single-precision floating point number
double	8	Double-precision floating point number
long double	16	Large double-precision floating point number

Note: any integer types are signed numbers. We can place the keyword “*unsigned*” in front of them to use them as positive numbers such as *unsigned char*, *unsigned short*, *unsigned int*, and *unsigned long*.

Statement

A statement is a complete direction that instructs a computer to carry out some task. In C programming, a semicolon (;) is used to mark the end of a statement. This rule is not applied to the preprocessor directives such as *#define* and *#include*. An example of

```
x = 2*a + b;
```

is an example of a statement. C programming is insensitive to the white spaces, including spacebar, tab, and newline. Thus, statements of

```
x=2 * a+b;
```

and

```
x =
    2 *
    a + b;
```

are all the same as the previous statement given above. If only a semicolon is given by itself on a line, it is called a “*null statement*”, which does not perform any action. Some may call it, “*No Operation*” statement.

Compound Statement

A compound statement is a block of two or more statements in a pair of braces, {...}. It is also called “statement block”. An example of compound statements is given below.

```
{
    int x = 10, y = 20;
    int z = x + y;
    printf(“%d + %d = %d\n”, x, y, z);
}
```

Expression

An expression is anything that evaluates to a numeric value. A simple expression may consist of a single item such as a simple variable, literal constant, or symbolic constant. An example of each simple expression is shown in Table 2-2.

Table 2-2. Simple expressions

Expression	Description
MAX	A symbolic constant
10	A literal constant
x	a variable

A symbolic constant evaluates to the value it is given in the *#define* directive. A literal constant evaluates to its own value. A variable evaluates to the value it is currently stored.

An expression may consist of a combination of more simple expressions, which normally are connected by one or more operators. This is called “complex expression”. The expression

$$x * 10$$

is composed of two simple expressions, a variable x and a literal constant 10 , which are connected by multiplication operator $*$. If the value of x is 2, then the expression $x * 10$ evaluates to 20. A complex expression may get even more complex as $x * x + x - 2$. In many cases, complex expression contains assignment operation such as

$$y = x + 10;$$

This statement first evaluates the expression $x + 10$ and then assigns the result to y . In this case, the assignment operator tells that the left side of the equal sign is to evaluate. Since this statement is also an expression, it evaluates to the value of y . If x is given as 10, the expression evaluates to 20.

Operator

An operator is a symbol that instructs C programming to perform some operation or action on one or more operands. An operand is an expression referring to something that an operator acts on. The fundamental operators in C programming include assignment operator and arithmetic operator.

Assignment operator

C programming uses equal sign, =, to indicate the assignment operator. A simple form of assignment is given as:

variable = expression;

which is that after the expression has been evaluated, the result will be assigned to the variable on the left side. An example of an assignment statement is

a = 10;

which means assigning the value of literal constant 10 to variable *a*. Or an assignment statement

of

a = b;

means assigning the value of *b* to *a*. (Keeping in mind that, in C programming, this is not an evaluation of whether *x* is equal to *y*.)

Arithmetic Operator

The arithmetic operators in C programming take two operands to perform a particular operation, which is called binary arithmetic operators. The common arithmetic operators are shown in Table 2-3.

Table 2-3. Arithmetic Operations

Integer Types	Floating Types
+ addition	+ addition
- subtraction	- subtraction
* multiplication	* multiplication
/ integer division: quotient	/ floating-point division
% integer division: remainder	

When $+$, $-$, and $*$ are applied to two integers, the operation is carried out using integer arithmetic, and the result is an integer. If either or both operands are floating-point numbers, the operation is carried out using floating-point arithmetic, and the result is a floating-point number

The effect of the division operation, $/$, differs for integer and floating-point operands. If both operands are integers, $/$ computes only the integer quotient; the remainder is discarded. Thus, the value of $9/2$ is 4. If either or both operands are floating-point numbers, a floating-point division is done. Thus, $9/2.0$, $9.0/2$, and $9.0/2.0$ are 4.5.

The $\%$ operation, *modulus*, computes the remainder of two integers. For example, $9\%2$ evaluates to 1, at which $9\%3$ evaluates to 0.

Arithmetic operators may take only a single operand. It is called unary arithmetic operators, which is shown in Table 2-4.

Table 2-4. Unary Arithmetic operators

Operator	Name	Action
$++$	Increment	Increase the value of operand by one
$--$	Decrement	Decrease the value of operand by one

The operand of unary arithmetic operators can only be an integer variable. The value of the operand is added or subtracted by 1. The statements of

```
 $x++;$ 
```

and

```
 $++x;$ 
```

are equivalent to the statement

```
 $x = x + 1;$ 
```

From the statements above, it can be seen that the unary operator can be placed before (*prefix*) or after (*postfix*) the operand. The difference of these two modes is when the value is incremented or decremented. The prefix operator will update the value after the operand before it is used while the postfix operator will update the operand value after it is used. The following statements are a good example of how the prefix operator works.

```
 $x = 10;$ 
```

```
 $y = x++;$ 
```

```
 $printf("x = \%d and y = \%d\n", x, y);$ 
```

In the *printf()* statement, the value of *x* is 11 while the value of *y* is 10. This is because when *x* is assigned to *y*, its value is equal to 10. After the assignment is done, the value of *x* is incremented by 1. As a result, the printed values are “*x = 11 and y = 10*”.

Precedence of Arithmetic Operators

In complex expressions, the order in which operation should be performed first is very important to the result from a calculation. Each operator has its own priority. The operator precedence is given in Table 2-5.

Table 2-5. The precedence of arithmetic operators

Operator	Precedence
++ --	1
* / %	2
+ -	3

From Table 2-5, it can be seen that unary operators have highest priority while the addition and subtraction have lowest priority. Thus the statement

$$x = 2 + 3 * 5;$$

will evaluate $3 * 5$ first. The result of the evaluation (15) is then added to 2 giving a result of 17 being assigned to *x*. However, in some cases, we may need to perform addition before multiplication. This can be done by enclosing the addition operation by parentheses. Therefore, the statement

$$x = (2 + 3) * 5;$$

will assign 25 to *x*.

Named Constant and Preprocessor Macro

In C programming, constants are named with the aid of *preprocessor macros* (usually just called *macros*). A macro is defined with the preprocessor directive *#define*, which is followed by the macro name and the desired definition. For example,

```
#define PI 3.14159265
```

defines *PI* to represent the floating number of 3.14159265.

Most header files contain macro definitions as well as function declarations. For example, the header file *stdlib.h* defines macros *EXIT_SUCCESS* and *EXIT_FAILURE* to represent the return codes for successful and unsuccessful execution, respectively. Thus, if *stdlib.h* has been included in the source file, we can replace

```
return 0;
```

by

```
return EXIT_SUCCESS;
```

and we can use

```
return EXIT_FAILURE;
```

to return the code for unsuccessful execution. An example of a C program is illustrated in Listing 2-2 below.

```
/* File: macro.c */
#include <stdio.h>
#include <stdlib.h>

#define MAX 100
#define MIN 0

int main()
{
    printf("Max - Min = %d\n", MAX-MIN);
    return EXIT_SUCCESS;
}
```

Listing 2-2. How to use preprocessor macro

Exercise

1. Write a directive that imports the library header file, `math.h`, into our program.
2. Write a command to compile a source file, `circle.c`, to generate an executable file named, `mycircle.exe`.
3. Write 10 valid identifiers for different types of variables.
4. Write a C program that computes area of a circle with radius of 5.

Chapter 3

Input and Output

Learning outcome

After completing this chapter, students should be able to:

1. Write a program to print data of each data type to computer screen
2. Write a program to receive data of each data type from keyboard
3. Print floating point numbers with assigned precision
4. Print formatted data to computer screen using field-width justification

The standard input/output library (`stdio.h`) provides a number of input and output functions. The most generally useful functions are *printf()* for output and *scanf()* for input. Function *printf()* displays formatted string to standard output (`stdout`), which is computer screen. Function *scanf()*, on the other hand, receives data of specified type from standard input (`stdin`), which is computer keyboard.

Output with *printf()*

From the previous chapter, we can see that *printf()* needs string as its argument to be printed to the computer screen. However, if we would like to print a number, we can simply add an additional argument to the function. Thus, the first argument of *printf()* is called *format string*, which can contain *conversion specifications* describing how other values are to be converted into printed form. The values to be converted are supplied as additional arguments. When the format string is printed, each conversion specification is replaced by the corresponding converted value.

A conversion specification always begins with a `%` sign. The simplest conversion specifications for the data types are listed in Table 3-1.

Table 3-1. Conversion specification for data type

Conversion Specification	Data Type
%d, %i	int
%f	float and double
%lf	double
%c	char
%s	char[] or char* (string)
%lu	unsigned int
%p	Address of a variable

It is also possible that a number may need to be represented in other forms. The conversion specification `%o` prints out numbers in octal decimal format, while `%x` represents numbers in hexadecimal format.

An example of `printf` constant number using conversion specification format is given as

```
printf("%d plus %d equals %d.\n", 20, 30, 50);
```

, which prints out as

20 plus 30 equals 50.

The arguments of `printf()` can be represented by variables or expressions. For example, the statements

```
x = 2.5;
y = 3.5;
printf("The product is %f.\n", x * y);
```

will print out

The product is 8.750000.

Field Width, Justification, and Precision

We can insert a number right after a conversion specification (%) to format the string to be printed. For example,

```
printf("%6d", 25);
```

will pad four spaces before 25. Hence, the printed value is right-justified in its field. Left-justified string can be done by placing a minus sign in front of the number. For example,

```
printf("%-10s", "Price");
```

prints *Price* followed by five spaces.

The field width can be followed by a decimal point and a precision. For example,

```
printf("%.3f", 8.75);
```

prints 8.750 right-justified in a seven-character field. Thus,

```
printf("%.3f", 8.75)
```

prints 8.750 in a five-character field.

The following program in Listing 3-1 illustrates an example of how to use conversion specifications to justify printed strings.

```
/* File: justify.c */
#include <stdio.h>
int main()
{
    printf("%-12s  %-7s\n\n", "Item", "Price");
    printf("%-12s  %-7.2f\n", "Computer", 1999.95);
    printf("%-12s  %-7.2f\n", "Typewriter", 249.98);
    printf("%-12s  %-7.2f\n", "Paper", 5.99);
    return 0;
}
```

Listing 3-1. Using conversion specification for print

The result from executing this program is shown below.

<i>Item</i>	<i>Price</i>
<i>Computer</i>	<i>1999.95</i>
<i>Typewriter</i>	<i>249.98</i>
<i>Paper</i>	<i>5.99</i>

Note: “\t can be used as a tab in the format string”.

Input with *scanf()*

The input function *scanf()* is similar to *printf()* in that input is controlled by a format string. The function *printf()* writes conversion specification to the standard output stream (*stdout*), while *scanf()* reads conversion specification from the standard input stream (*stdin*), which is connected to the user's keyboard.

One distinction is that *printf()* takes variable as its argument while *scanf()* takes *the address of variable*, which is represented as an ampersand sign, &, preceding the variable. For example, the statements

```
int x;
scanf("%d", &x);
```

reads an integer number and stores it in the variable *x*. Likewise, the statement

```
scanf("%d%d%d", &i, &j, &k);
```

reads three integers from standard input stream, converts them into type *int*, and stores them in variables *i*, *j*, and *k*, respectively. Values to be read are separated by whitespace, which can consist of spaces, tabs, and newlines.

String format of *scanf()* uses the same conversion specifications as ones used in function *printf()*.

```
%d for int
%c for a char
%f for float
%lf for double
%s for char[ ] or char*
```

Here is an example of how to use conversion specification to read data from standard input stream (*stdin*) into a variable of its type.

```

char c;
scanf("%c", &c);
int x;
scanf("%d", &x);
float f;
scanf("%f", &f);
char str[10];
scanf("%s", str);

```

From the example above, we can see that *scanf()* takes the variable's address as its argument for all data types except the array of characters (string). In fact, variable *str* represents the beginning address of an array of characters. Alternatively, we can use *&str[0]* as the beginning address of the array. Listing 3-2 displays a program that receives data from *stdin* (keyboard) and prints them with formatting on the computer screen.

```

/* File: input.c */
#include <stdio.h>
int main() {
    char name[40];
    int age;
    char grade;
    float gpa;
    printf("Enter your name: ");
    scanf("%s", name);
    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter C programing grade: ");
    scanf(" %c", &grade); //Make sure to precede a spacebar
                          //in conversion specification
    printf("Enter your GPA: ");
    scanf("%f", &gpa);

    /*** Display the entered information ***/
    printf("%10s: %s\n", "Name", name);
    printf("%10s: %d\n", "Age", age);
    printf("%10s: %c\n", "C Grade", grade);
    printf("%10s: %.2f\n", "GPA", gpa);
    return 0;
}

```

Listing 3-2. Input with *scanf()*

Please note that the format string of reading character contains a space in front of conversion specification `%c`.

More on Output Stream Functions

C language provides other functions to write a single character to standard output stream (computer screen) as follows.

```
int putc(int char, FILE *stream);
```

Function *putc()* takes a character code (ASCII) and a specific stream to write the character out as its arguments. In case of displaying on the computer screen, *stdout* is given. Upon successful call, it returns the character code it has written to. *EOF* is returned otherwise.

```
int putchar(int char);
```

Function *putchar()* displays a character code, specified as its argument, to the computer screen. Upon successful call, it returns the character code it has written to. *EOF* is returned otherwise.

Both statements below will print out a character, *A*, to the computer screen.

```
putc('A', stdout);  
putchar('A');
```

Instead, if a string is to be printed to the computer screen, *puts()* is then needed.

```
int puts(const char *str);
```

Function *puts()* displays a string to the *stdout* (computer screen). A newline character will be appended to the output. It returns a non-negative number if the call is successful. Otherwise, it returns *EOF*. The following statements will print “Hello” to the computer screen.

```
char str[10] = “Hello”;  
puts(str);
```

More on Input Stream Functions

C language provides other functions to read a single character from standard input stream (keyboard) as follows.

```
int getc(FILE *stream);
```

It reads a character at a time from an input stream and returns an integer value representing ASCII code, which will be converted to *char* type. In case of failure, *getc()* returns *EOF*. Since we need to read a character from the standard input stream (keyboard), *stdin* has to be passed as the argument stream on *getc()*.

```
int getch();
```

Function *getch()* reads a single character from the keyboard. It returns ASCII code immediately without pressing the Enter key. However, the character that is entered will not appear on the computer screen.

```
int getche();
```

Function *getche()* works as *getch()* does except that the character that is entered will appear on the computer screen.

```
int getchar();
```

Function *getchar()* also reads a single character from the keyboard. It returns ASCII code after the Enter has been pressed. The entered character is displayed on the computer screen.

The following statements illustrate how to use these functions, given variable *c* declared as *char* type.

```
char c;
```

```
c = getc(stdin);
```

```
c = getch();
```

```
c = getche();
```

```
c = getchar();
```

Instead, if a string is to be received from a keyboard, function *gets()* is needed.

```
char *gets(char *str);
```

Function *gets()* reads a stream of data from the keyboard and puts it in a string variable (array of characters). On success, it returns the string received from the keyboard. Otherwise, it returns *NULL*. The following statements illustrate how to read a string to a variable using *gets()*.

```
char name[30];  
gets(name);
```

Listing 3-3 demonstrates how to use *get* and *put* functions to read a character from the keyboard and write it to the computer screen, while Listing 3-4 demonstrates how to work on string.

```
/* File: getc_putc.c */  
#include <stdio.h>  
int main()  
{  
    char c;  
    printf("a : Add\n");  
    printf("u : Update\n");  
    printf("d : Delete\n");  
    printf("Please select: ");  
    c =getch();  
  
    printf("You select: ");  
    putchar(c);  
  
    return 0;  
}
```

Listing 3-3. Getting a character from keyboard and put it on screen

```

/* File: gets_puts.c */
#include <stdio.h>
int main()
{
    char name[30];
    printf("Please enter your name: ");
    gets(name);

    puts("Hello ");
    puts(name);

    return 0;
}

```

Listing 3-4. Getting string from keyboard and put it on screen

Exercise

1. Write a program to receive two values of integer from keyboard to perform addition, then print the result on computer screen
2. Write a program to receive two values of double from keyboard and print the result of addition, subtraction, multiplication, and division on computer screen, at which the results are printed with the precision of 2 digits
3. write a program to receive data from keyboard as follows:

name - of type char[40]
age - of type int
score - of type double
grade - of type char

and print them out using field width justification to format the result.

Chapter 4

Control Statement

Learning outcome

After completing this chapter, students should be able to:

1. Write a program using *if-else* statement to classify given items
2. Write a program using *switch-case* statement to make selection from given options
3. Write a program using conditional operator to classify given items

A control statement tests the state of the computation by evaluating a control expression; each control expression tests whether or not a certain condition is satisfied. The value of the expression represents the result of the test and determines the action of the control statement.

For most control statements, the result of a test can be represented as *true* or *false*. In C programming, zero represents *false* and any nonzero values represent *true*. The relational operators are presented in the Table 4-1.

Table 4-1. Relational Operators

Operator	Condition
==	is equal to
!=	is not equal to
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to

Selection Statement

C language has three selection statements: the *if* statement, the *if-else* statement, and the *switch* statement.

The *if* Statement

The *if* statement has the form:

```
if (expression)
    statement;
```

The control expression is evaluated first. If the value is *true*, the controlled statement is then executed. If the control expression is *false*, the controlled statement is ignored. See an example:

```
if (count > MAX_COUNT)
    printf("Overflow\n");
```

An interesting example is given in Listing 4-1, on which indentations are used in trying to group statements and increasing readability of the program.

```
/* File: if.c */
#include <stdio.h>
int main() {
    int score = 49;
    if(score >= 50)
        printf("You pass.\n"); //Controlled statement
        printf("You are a good student.\n"); //Normal statement
    return 0;
}
```

Listing 4-1. *if* statement

As mentioned earlier, statement *if* requires only one controlled statement after the expression. Therefore, only the statement that prints out “*You pass.*” is skipped if the expression is evaluated as *false*. “*You are a good student.*” will always be executed no matter what happens to the *if* statement. In this case, if multiple controlled statements need to be executed, they have to be grouped into a block as shown in the example below.

```
if (count > MAX_COUNT)
{
    printf("Overflow\n");
    count = 0;
}
```

Thus, the program in Listing 4-1 should be fixed as shown in Listing 4-2

```

/* File: if_block.c */
#include <stdio.h>
int main() {
    int score = 49;
    if(score >= 50) {
        printf("You pass.\n");
        printf("You are a good student.\n");
    }
    return 0;
}

```

Listing 4-2. Statement *if* with block of statements

From Listing 4-2, the *if* statement checks for an examination score that is greater than or equal to 50. If satisfied, this program will print out “*You pass.*”, followed by “*You are a good student.*”. Otherwise, it does nothing. What we can see from this is that this program ends its execution without doing anything. The *if* statement is a one-way test. If we want to check for the alternative, we have to add another *if* statement to do so. Listing 4-3 illustrates a two-way test, which solves the above problem.

```

/* File: multiple_if.c */
#include <stdio.h>
int main() {
    int score = 50;
    if(score >= 50) {
        printf("You pass.\n");
        printf("You are a good student.\n");
    }
    if(score < 50) {
        printf("You fail.\n");
        printf("Let's study harder.\n");
    }
    return 0;
}

```

Listing 4-3. Multiple *if* statement

The *if-else* Statement

According to the problem in the previous section, C language actually provides two-way test for *if* statement as *if-else* statement. Adding an *else* to *if* statement gives us the *if-else* statement as shown in the following.

```
if (expression)
    statement-1
else
    statement-2
```

If the value of the control expression is evaluated as *true*, *statement-1* is executed. If it is *false*, *statement-2* is executed as the following example:

```
if (count < MAX_COUNT)
    count = count + 1;
else {
    printf("Overflow\n");
    count = 0;
}
```

Thus, a better version of the program in Listing 4-3 is given in Listing 4-4.

```
/* File: if_else.c */
#include <stdio.h>
int main()
{
    int score = 50;
    if(score >= 50) {
        printf("You pass.\n");
        printf("You are a good student.\n");
    } else {
        printf("You fail.\n");
        printf("Let's study harder.\n");
    }
    return 0;
}
```

Listing 4-4. *if-else* statement

Nested *if* and *if-else* Statement

The controlled statement in *else* can itself be an *if* or *if-else* statement. This results in nested *if-else* statements as shown in example below.

```

if (expression-1)
    statement-1
else if (expression-2)
    statement-2
else if (expression-3)
    statement-3
else
    statement-5

```

In this case, the expression-1 is evaluated. If this is true, statement-1 will be executed. Then, the rest (from *else* on) will be discarded. On the other hand, if the expression-1 is evaluated as false, its associated *else* is executed, at which its controlled statement is another *if-else* statement. This requires the expression-2 to be evaluated before deciding whether statement-2 should be executed. This will go on and on until either will be satisfied. The following code in Listing 4-5 uses nested *if-else* to check the number of decimal digits in a positive integer *n*.

```

/* File: nested_if.c */
#include <stdio.h>
int main() {
    int n;
    printf("Please enter number: ");
    scanf("%d", &n);
    if (n < 10)
        printf("One digit\n");
    else if (n < 100)
        printf("Two digits\n");
    else if (n < 1000)
        printf("Three digits\n");
    else
        printf("Four or more digits\n");
    return 0;
}

```

Listing 4-5. Nested *if* statement

Note: if multiple controlled statements are needed, they have to be in a block.

The *switch-case* and *break* Statement

Another approach to multiway selection is the *switch* statement, which has the following general form as:

```
switch (expression)
  statement
```

The controlled statement is normally a block, one or more of whose statements are preceded by *case labels* as an example:

```
switch (2) {
  case 1:
    statement-1
  case 2:
    statement-2
  case 3:
    statement-3
}
```

Since the expression in *switch()* is evaluated as 2, the execution of the block begins after the label *case 2:*, rather than the beginning of the block. If 5 is given as an expression, *switch(5)*, this would cause the execution to skip all cases.

The case label *default:* corresponds to any values of the control expression that are not represented by any *case* label. Thus in

```
switch (5) {
  case 1:
    statement-1
  case 2:
    statement-2
  case 3:
    statement-3
  default:
    statement-default
}
```

statement-default will be executed because none of the other *case* labels corresponds to the value of the control expression. Listing 4-6 is an example of how a *switch* statement works.

```
/* File: switch.c */
#include <stdio.h>
int main()
{
    switch(1) { // Let's try to change to some other number
        case 1:
            printf("One\n");
        case 2:
            printf("Two\n");
        default:
            printf("Default\n");
    }

    return 0;
}
```

Listing 4-6. Statement *switch-case*

The problem of the program in Listing 4-6 is that once a case is matched, the corresponding state is then executed. Then, all the following statements in the following cases will also be executed. In fact, after executing the statements of the selected case, we want to leave the block and continue with the rest of the program. We accomplish this with a *break* statement, which directs the computer to exit from a *switch* statement. Thus, the statements for each *case* should end with a *break* statement as illustrated in Listing 4-7.

```

/* File: switch_break.c */
#include <stdio.h>
int main()
{
    int x=1; // Let's try to change to some other number

    switch(x) {
        case 1:
            printf("One\n");
            break;
        case 2:
            printf("Two\n");
            break;
        default:
            printf("Default\n");
            break; // in fact, this break is not necessary
    }

    return 0;
}

```

Listing 4-7. Statement *switch-case-break*

In Listing 4-7, after “One” has been printed, the execution will exit the *switch* statement immediately. Sometimes, we may need *switch* to match multiple cases. For instance, a program requires user to enter a single character for a menu selection. User may enter either a lowercase or uppercase character. Listing 4-8 demonstrates how to use *case-break* to allow both lowercase and uppercase characters to represent the same option.

```
/* File: double_case.c */
#include <stdio.h>
int main()
{
    char c;

    printf("Enter a or b : ");
    c = getch();

    switch(c) {
        case 'a':
        case 'A':
            printf("You select a or A.\n");
            break;
        case 'b':
        case 'B':
            printf("You select b or B.\n");
            break;
        default:
            printf("You select something else.\n");
    }

    return 0;
}
```

Listing 4-8. *case* with multiple match

Conditional Operator

C Language provides a short *if-else* statement in the form of a conditional operator. The conditional operator has three operands, which are separated by the symbols ? (question mark) and : (colon) as shown below:

$$exp-c \ ? \ exp-t \ : \ exp-f;$$

exp-c is the control expression; its value determines which of the two following expressions will be evaluated. If *exp-c* yields *true*, *exp-t* is evaluated. If *exp-c* yields *false*, *exp-f* is evaluated. For example,

```
int x;
x = m >= n ? m : n;
```

is grouped as

$$x = ((m \geq n) ? m : (n));$$

From the above statement, *x* will be assigned by either the value of *m* or *n* depending on if *m* is greater than or equal to *n*, then *m* will be assigned to *x*. Otherwise, *n* will be given. This is a simple way to determine which has greater value.

Another example is given below. (Suppose *n* is an unsigned integer representing the number of cars we own):

```
printf("I have %d %s.\n", n, (n<2)? "car": "cars");
```

The result of this statement is shown as:

- If *n* is 2 or more, the result would be:

I have 2 cars.

- If *n* is 1 or 0, the result would be:

I have 1 car. Or I have 0 car.

Exercise

1. Write a C program that receives a student score from keyboard then uses *if* statement to print out a grade based on the following criteria:

Range	Grade
80 - 100	A
70 - 79	B
60 - 69	C
50 - 59	D
0 - 49	F

2. Write a C program that receives a student score from the keyboard, and then uses *conditional operator* to print out a grade based on the given criteria in Question 1.
3. Use *switch-case* statement for writing a C program to select an operation via keyboard according to the following menu

Menu

1. Insert data
2. Edit data
3. Delete data
4. Search data
5. Quit

Chapter 5

Iteration

Learning outcome

After completing this chapter, students should be able to:

1. Write a program using an iterative statement to perform repeated operations
2. Write a program to iterate operation upon given condition

One of basic operations in computer programming is an ability to perform its task repeatedly. A program can perform millions of iterations without taking a break. In addition, it remains its execution without fatigue. This feature makes computers superior to human beings. Iteration in C programming is accomplished with the *while*, *do*, and *for* statements.

The *while* Statement

The *while* statement has the following form:

```
while (expression)
    statement
```

Expression is the control expression and *statement* is the controlled statement. If the value of the control expression is *true* (or nonzero), the controlled statement is executed, after which the control expression is evaluated again, and so on. If the value of the control statement is *false* (or zero), the controlled statement is not executed and the execution of the *while* statement is terminated.

The following example demonstrates a *while* statement that runs 10 iterations.

```
n = 1;
while (n <= 10)
    n = n + 1;
```

The statements above iterate the increment of n as long as n is less than or equal to 10. It keeps repeating until n reaches 11, which is greater than 10. This causes the control expression to be evaluated as *false*, resulting in the termination of the *while* statement. However, if multiple controlled statements are needed, they can be grouped in a block as well. In Listing 5-1, the controlled statements print the value of n followed by the increment of n .

```

/* File: while.c */
#include <stdio.h>
int main()
{
    n = 1;
    while (n <= 10)
    {
        printf("%3d", n);
        n = n + 1;
    }
    return 0;
}

```

Listing 5-1. Iteration with *while* statement

The above statements in Listing 5-1 produce the following printout:

```
1 2 3 4 5 6 7 8 9 10
```

The statement *while* is suitable for iterations that run infinitely until a specific condition is satisfied.

The *do-while* Statement

The *do-while* statement is another form of iteration. It is similar to what the *while* statement does. The difference of these two is that the *while* statement evaluates control expression before executing controlled statement(s), whereas the *do-while* statement will at least execute the controlled statement once, then evaluates control expression. The *do-while* statement has the following form:

```

do
    statement
while (expression);

```

In the following example, the *while* statement will not be evaluated until a number has been read from the keyboard and printed to the computer screen:

```
do {
    printf("Please enter number: ");
    scanf("%d", &n);
    printf("You enter: %d\n", n);
} while (n < 1000);
```

The *do-while* statement gives the controlled statements a chance to perform at least once, then it decides whether the controlled statements should be repeated. Listing 5-2 illustrates how a *do* statement is to be used.

```
/* File: menu.c */
#include <stdio.h>
int main()
{
    int x;

    do {
        printf("Menu: \n");
        printf(" 1. Add \n");
        printf(" 2. Update \n");
        printf(" 3. Delete \n");
        printf(" 4. Search \n");
        printf("Please select 1-4: ");
        scanf("%d", &x);
    } while(x<1 || x>4); // repeat if x is not 1-4.

    // perform the selected action

    return 0;
}
```

Listing 5-2. Menu selection checking with *do-while* statement

Listing 5-2 allows user to select an action (1-4). If the user enters a value which is not between 1 and 4, the program will keep asking until the user enters one of the provided actions.

The *for* Statement

The *for* statement is suitable for iterations that have a definite number of loops. The *for* statement has the following form:

```
for (exp-i; exp-t; exp-s)
    statement
```

Expression *exp-i* assigns the control variable its initial value, *exp-t* tests whether the iterations should continue, and *exp-s* steps the control variable from one value to the next. An example of using a *for* statement to print a number from 1 to 10 is given below.

```
for (n=1; n<=10; n=n+1)
    printf("%3d", n);
```

From this statement, *exp-i* (*n = 1*) assigns *n* the initial value 1, *exp-t* (*n <= 10*) ensures that the value of *n* will not exceed 10, and *exp-s* (*n = n + 1*) increments the value of *n* after each execution of the controlled statement (*printf()*). The above *for* statement is equivalent to the following *while* statement:

```
n = 1;
while (n<=10) {
    printf("%3d", n);
    n = n + 1;
}
```

which is in the form of:

```
exp-i;
while (exp-t) {
    statement
    exp-s;
}
```

Listing 5-3 demonstrates how to use a *for* statement to compute a summation of integer numbers taken from *stdin* (keyboard).

```

/* File: sum.c */
#include <stdio.h>
int main() {
    int number, sum = 0;
    for(n=1; n<=10; n=n+1) {
        scanf("%d", &number);
        sum = sum + number;
    }
    printf("sum = %d\n", sum);
    return 0;
}

```

Listing 5-3. Summation with *for* statement

Compound Assignment Operator

A compound assignment operator consists of a C operator followed by an equal sign (=), as in +=, -=, *=, and /=. Specifically, for the arithmetic operators, we have

$m += n$	is equivalent to	$m = m + n$
$m -= n$	is equivalent to	$m = m - n$
$m *= n$	is equivalent to	$m = m * n$
$m /= n$	is equivalent to	$m = m / n$
$m \% = n$	is equivalent to	$m = m \% n$

An example of how to use a compound assignment operator is given as

$m *= n+5;$, which is grouped as $m *= (n+5);$

The above statement is to add 5 to n first, then multiply with m , and finally store the result to the variable m itself.

From the example in Listing 5-3,

$sum = sum + number;$

could have been written more compactly and idiomatically as

$sum += number;$

Increment and Decrement Operators

We already know that $n = n + 1;$ can be written more compactly as $n += 1;$. Therefore, the following three expressions are equivalent:

```
n = n + 1;
n += 1;
++n;
```

Likewise, the following expressions are also equivalent:

```
n = n - 1;
n -= 1;
--n;
```

The increment and decrement operators can be used as *prefix* operators (as in $++n$ and $--n$) or as *postfix* operators (as in $n++$ and $n--$). The prefix operator will increment or decrement the value of a variable and then return a new value to the variable as the value of expression. With the postfix operator, the value of the expression is the *old* value of the variable (the value before increment/decrement), then the value of the variable is incremented/decremented. For example, the statements

```
int n = 25;
```

prefix operator	postfix operator
$m = ++n;$	$m = n++;$

The results from both operators are not equivalent. Both will increment the value of n but the value of m in the left column, *prefix operator*, will be 26 (increment the value of n first then assign to m), while the value of m in the right column, *postfix operator*, remains 25 (old value of n). However, after the assignment to m has been executed, the value of n will become 26 (increment after).

Note: the assignment statements above can be grouped as

```
m = (++n);
m = (n++);
```

,which yield the same results as the example above.

An example of integer summation showing how to use both $++$ (*increment operator*) and $+=$ (*compound assignment operator*) is given in Listing 5.4.

```

/* File: new_sum.c */
#include <stdio.h>
int main() {
    sum = 0;
    for(n = 1; n <= 10; n++) {
        scanf("%d", &number);
        sum += number;
    }
    printf("sum = %d\n", sum);
    return 0;
}

```

Listing 5-4. New summation using *for* statement

Exercise

1. Write a program that asks user to enter a positive integer number for printing a multiplication table.
2. Write a program that receives a character from user via the keyboard and repeats this process until 'q' is entered.
3. Write a program that prints out factorial numbers from 0! to 10!
4. Write a program to display Fibonacci numbers from a number given via the keyboard.

Chapter 6

Array Variable

Learning outcome

After completing this chapter, students should be able to:

1. Write a program to operate on array of data
2. Write a program using iterative statement to process array of data

In many cases, we need multiple variables of the same type to perform a specific task. For example, we need to compute an average value of students' from a final examination. If there are 30 students in a class, 30 variables are needed to store all students' score. It is not wise to allocate 30 individual variables such *int x1, x2, x3,.....* and so on. Many programming languages, including C language, provide the array of variables to allocate and access memory for multiple variables sharing the same name and same data type.

One-Dimensional Array

A one-dimensional array is a series of objects, all of which have the same type. These objects are the elements of the array. In other words, an array is a block of memory made up of memory locations of a fixed size. In C language, we declare an array by giving the name of the array with its type and a number of elements. For example,

```
int a[5];
```

declares *a* as an array of five *int* objects. Likewise,

```
long double x[100];
```

declares *x* as an array of 100 *long double* objects.

We can initialize an array when it is declared by listing the initial values for its elements in curly braces, as follows:

```
int a[5] = {75, 25, 100, -45, 60};
```

An array with five elements has subscripts (array indexes) ranging from 0 to 4. The values assigned to the array *a* can be illustrated in figure 6-1:

a[0]	75
a[1]	25
a[2]	100
a[3]	-45
a[4]	60

Figure 6-1. Array variable with values

A declaration (with initialization) of

```
int b[5] = {75, 25};
```

is equivalent to

```
int b[5] = {75, 25, 0, 0, 0};
```

Subscripted array can be used in the same way as other typical variables, such as *x*, *score*, *discount*, *interest*, etc. For example,

```
a[2] = 5;
```

assigns 5 to element 2 (the third element) of *a*, while

```
n = a[i] - a[j];
```

assigns *n* with the difference value of the *i*-th and *j*-th of *a*, and

```
a[2*i - j]++;           or           ++a[2*i - j];
```

increments the value of the element $(2 * i) - j$. (see postfix and prefix operators)

Constants represented by macros are frequently used in declaring and manipulating arrays. For example, if *COUNT* is defined by the directive

```
#define COUNT 1000
```

then

```
int c[COUNT];
```

declares variable *c* as an array of 1000 *int* objects.

Subscripts (array index) for *c* programming range from 0 through *COUNT-1*. Typically, a *for* statement is used for processing all the elements of *c* as shown below.

```
for(i = 0; i < COUNT; i++) {
    /* Process element c[i] */
}
```

An example of setting all elements to 0 is given below

```
for(i = 0; i < COUNT; i++)
    c[i] = 0;
```

An equivalent implementation using *while* statement is given as:

```
int i = 0;
while(i < COUNT)
    c[i++] = 0;
```

This above example uses postfix increment operator to increase the array index, *i*, by 1 after 0 has been assigned to the current *i-th* value of *c*.

Array and *for* Statement

The nature of the array variable is that it contains multiple values, at which a specific value can be accessed via its index. Accessing multiple consecutive values from an array is made possible by iterative statements. The most common way to access each value of an array variable is to use the *for* statement, which can easily set a number of loops it wants to iterate. Listing 6-1 illustrates how to assign a value to each object of an array via *stdin* (keyboard) using *for* statement. Subsequently, each value is printed out also using *for* statement.

```
/* File: array_for.c */
#include <stdio.h>
int main()
{
    int i, num=10;
    int data[10];

    for(i=0;i<10;i=i++)
    {
        printf("x[%d] : ", i);
        scanf("%d", &data[i]);
    }

    for(i=0;i<10;i=i++)
        printf("x[%d] = %d\n", i, x[i]);

    return 0;
}
```

Listing 6-1. Array with *for* statement

Listing 6-2 illustrates how to use a *for* statement to read all values from an array of students' scores. Then, all read values are used to compute a summation value and the average value respectively.

```
/* File: array_average.c */
#include <stdio.h>
#define SIZE 10;
int main()
{
    int score[SIZE] = { 4, 7, 9, 10, 6, 8, 5, 9, 8, 5 };
    int i, sum=0;
    double avg;

    for(i=0; i<SIZE; i++)
    {
        sum += score[i];
    }
    printf("Sum of score = %d\n", sum);

    avg = (double) sum/SIZE;
    printf("Average = %.2f\n", avg);

    return 0;
}
```

Listing 6-2. Computing summation and average values from array variable

Exercise

1. Write a program that ascendingly sorts values in an array of integers and displays the result on the computer screen.
2. Write a program that descendingly sorts values in an array of integers and displays the result on the computer screen.
3. Write a program that prints out all grades of 10 students' scores stored in an array of integers. (The grade criteria is given in the table below).

Range	Grade
80 - 100	A
70 - 79	B
60 - 69	C
50 - 59	D
0 - 49	F

Chapter 7

Pointer and String

Learning outcome

After completing this chapter, students should be able to:

1. Use indirection operator to refer to value of target object
2. Tell the difference between value of pointer variable and value of typical variable
3. Use pointer variable and array variable interchangeably
4. Use pointer-to-char variable to manipulate string value

A *pointer* is the address of an object of a *particular type*. An address only locates the first byte of an object. We need to know the type because it can determine the size of the object, which is the number of bytes stored in it. Typical pointer types are *pointer-to-int*, *pointer-to-double*, *pointer-to-char*, and so on.

Indirection Operator

If p is a variable with type *pointer-to-int*, the value of p is the address of an *int* object. The indirection operator, $*$, enables us to refer to the target object. Thus, $*p$ is an lvalue of an object that is pointed by p . (See Figure 7-1)

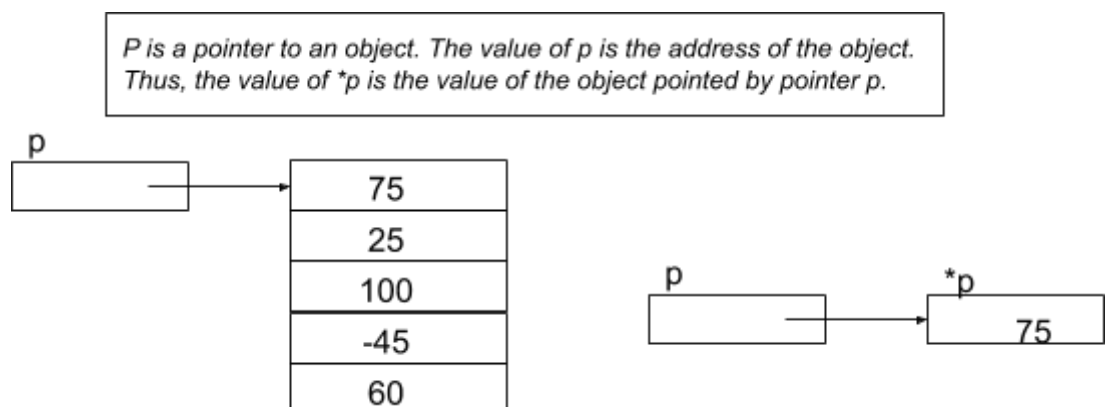


Figure 7-1. Pointer to an object

The *indirection* operator is a prefix unary operator, which can be distinguished from the binary *multiplication* operator because the indirection operator is applied to only one operand whereas the *multiplication* operator is always applied to two operands.

From Figure 7-1 above, **p* behaves like a typical *int* variable with value of 75. For example,

```
printf("%d", *p);
```

prints the value 75 to the computer screen, and

```
n = *p + 10;
```

assigns 85 to *n*, while

```
*p = 20;
```

assigns 20 to the **p*, and

```
*p += 7;    is equivalent to    *p = *p + 7;
```

,which adds 7 to **p*, then stores the result to **p*, whereas

```
(*p)++;
```

increments the value of the target object (*postfix* increment).

We declare a pointer variable by giving the type of the target object. For example,

```
int *p;
double *q;
```

declare *p* with the type of *pointer-to-int* and *q* with the type of *pointer-to-double*.

Note: Only declaring a pointer variable does not make it point to any particular object. We must not attempt to apply the *indirection operator* to a pointer value until the variable has been assigned with the address of a valid object.

Using *Address-Of* Operator

We can use the *address-of* operator, `&`, to compute the address of any object designated by an lvalue. Such addresses have the pointer types and can be used to initialize and assign values to the pointers. For example, if we declare

```
int n = 100;
double x = 3.5;
```

then `&n` has the type of *pointer-to-int* and `&x` has the type of *pointer-to-double*. If we now declare

```
int n = 100;
int *p = &n;
```

then `p` is initialized to point to `n`. Thus, `*p` and `n` use the same object (they are said to be *aliases*), at which both have the value of 100. In other words, both `*p` and `n` share the same memory location. Thus, *any changes applied to *p or n will affect the other*. If we now declare

```
double x = 20.0;
double *q;
```

then `q` does not point to a valid object. This can be fixed by assigning

```
q = &x;
```

`q` now points to the object named by `x`. If we attempt to change the value of `*q` to 7.9, as follow:

```
*q = 7.9;
```

`x` will then have the value of 7.9 as well. (This is because both `x` and `*q` share the same memory location to store their value.)

Using *const* in Pointer Declaration

In a pointer declaration, keyword *const* can be used in two different places, with two different meanings. If *const* is placed at the beginning of the declaration, the target object is declared constant. For example, if we declare

```
const int *p;
```

then the value of the target object **p* cannot be changed with the assignment, increment, and decrement operators. Thus, statements such as

```
*p = 100;
```

and

```
(*p)--;
```

are *invalid*. However, the use of *address-of* operator to *p* is legal. For example, if *m* and *n* are integer variables, then the statements

```
p = &m;           and           p = &n;
```

can be applied to *p*.

On the other hand, if *const* precedes the pointer variable, then the value of *p* (address of which *p* points to) now becomes constant. In this case, **p* can be freely changed. For example, if we declare

```
int *const p = &m;
```

then *p* will retain its initial value of *&m*, Assignment and increment such as

```
p = &n;  
p++;
```

are *not* allowed. On the other hand, we can freely assign new value to **p*, including increment and decrement **p* as follow:

```
*p = 25;  
*p += 3;  
(*p)++;  
(*p)--;
```

Pointer and Array

We can use the name of an array to initialize a pointer to the address of the first element of the array. For example,

```
int list[100];
int *p = list;
int *q;
q = list;
```

both *p* and *q* point to the first element of the variable *list*. In fact, C language allows the *subscript* operator to be applied to pointer as well as array names. Hence, it is legitimate to use *p[0]* to refer to the first element of the variable *list*. It is also possible to refer to the 5th element of *list* by using *p[4]*. As a result,

```
sum = 0;
for(i = 0; i < 100; i++)
    sum += list[i];
```

can also be written as

```
sum = 0;
for(i = 0; i < 100; i++)
    sum += p[i]; // also sum += q[i]
```

A pointer is not always necessary to point to the first element of an array. For example, we can set *p* to point to element 50 of *list* (as shown in Figure 7-2):

```
p = &list[50];
```

Now, *p[0]* is an alias of *list[50]*, *p[1]* is an alias of *list[51]*, and so on. Likewise, *p[-1]* is an alias of *list[49]* while *p[-2]* is an alias of *list[48]*, and so on.

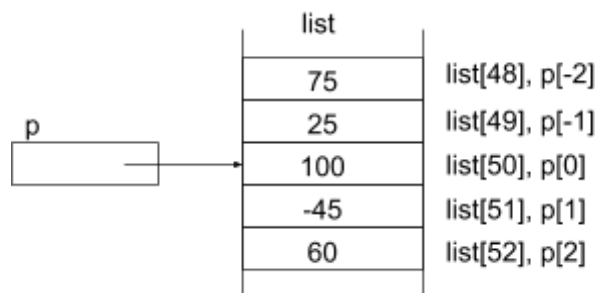


Figure 7-2. Pointer to array of data

It is very important to keep in mind that the subscript (index) of an array must be an integer number and it has to be a positive number. However, what we can see from Figure 7-2 is that the subscripts given for *p* (which is actually declared as *pointer-to-int* rather than an array of integers) are negative numbers. This is legitimate to use on pointer variables but is not allowed on array variables. Nevertheless, the most important thing is that *p must point to a valid location of the target object.*

Listing 7-1 displays a use of a pointer variable to read and write integer values while Listing 7-2 displays a use of a pointer variable to access an array variable.

```

/* File: pointer2int.c */
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;
    printf("x = %d - *p = %d\n", x, *p);

    *p = 20;
    printf("x = %d - *p = %d\n", x, *p);

    x = 30;
    printf("x = %d - *p = %d\n", x, *p);
    return 0;
}

```

Listing 7-1. Pointer to integer variable

```

/* File: pointer2array.c */
#include <stdio.h>
int main() {
    int x[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int *p = x;
    int i, sum=0;
    for(i=0; i<10; i++)
        sum = sum + *(p+i); // equivalent to p[i]
    printf("Sum = %d\n", sum);
    return 0;
}

```

Listing 7-2. Pointer to array

String

In C programming, strings are stored in memory as an array of *char* values. Each character of a string is stored in each byte of the array of *char*. To indicate where a string ends, the final character of the string is followed by a *null* character (`'\0'`), whose code is zero. We can show that a *null* character terminates a string by executing

```
printf("abcd\0efgh");
```

As a result, the computer prints only

abcd

on the screen because the *null* character indicates the end of the string after 'd'. We can declare an array *str* to hold strings of up to three characters as follows:

```
char str[4];
```

Note: the number of array elements should be one more than the number of characters, to provide room for the terminating *null* character. The following declaration initializes *str* with the string “dog” as

```
char str[4] = "dog";
```

which can also be written as

```
char s[4] = {'d', 'o', 'g', '\0'};
```

and can be illustrated as shown in Figure 7-3:

s	
s[0]	'd'
s[1]	'o'
s[2]	'g'
s[3]	'\0'

Figure 7-3. Array of characters

However, we can also leave it to the compiler to determine the number of array elements for a string as

```
char str[] = "dog";
```

The declaration of

```
char str[10] = "dog";
```

results in an allocation of ten characters at which only four of them are actually used to store the string as shown in Figure 7-3.

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
'd'	'o'	'g'	'\0'						

Figure 7-3. Assigning string to array of character

If *str* has been declared and initialized as in any of the preceding examples, then

```
printf(s);
```

and

```
printf("%s", s);
```

will yield the same result of printing “dog” to the computer screen.

Array names and string literals can also be used to initialize and assign values to pointers. Thus

```
char *p = str;
```

initializes *p* to point to the first character of *str*, and

```
char *p = "dog";
```

initializes *p* to point to the first character of “dog”. Since strings are passed to functions as pointers, the statement

```
printf(p);
```

also causes the computer to print “dog” to the screen. Listing 7-3 uses an array of *char* and *pointer-to-char* variables to receive string values from the keyboard and print them to the computer screen.

```
/* File: string_input_output.c */
#include <stdio.h>
int main()
{
    char str[20];
    char *p = str; // p and str share same memory location

    printf("Please enter your firstname: ");
    scanf("%s", str);
    printf("Your name is %s\n", str);
    printf("Hello %s\n", p); // p also holds value of str

    printf("Please enter your lastname: ");
    scanf("%s", p);
    printf("Your lastname is %s\n", str); //str also holds value of p
    printf("Hello %s\n", p);

    return 0;
}
```

Listing 7-3. Getting string from keyboard and writing to screen

Exercise

```
int x = 10;
float f = 10.5;
double d = 4.0;
char c = 'A';
```

1. Use the above statements to answer the following questions

- what is the address of x ($\&x$) ?
- what is the address of f ($\&f$) ?
- what is the address of d ($\&d$) ?
- what is the address of c ($\&c$) ?

```
int x = 20;
int* p = &x;
*p += 1;
```

2. Use the above statements to answer the following questions

- what is the address of x ($\&x$) ?
- what is the address of p ($\&p$) ?
- what is the value of p ?
- what is the value of $*p$?
- what is the value of x ?

$x[0]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$x[5]$	$x[6]$	$x[7]$	$x[8]$	$x[9]$
0	10	20	30	40	50	60	70	80	90

3. Given the following statements

```
int* p = &x[4];
printf("%d %d %d\n", *(p-4), *p, *(p+5));
printf("%d %d %d\n", p[-2], p[0], p[3]);
```

- write the result from the first `printf()`
- write the result from the second `printf()`

4. Giving a string

```
str[] = "We always have fun once we start writing a C program. This makes  
us so happy";
```

Write a program that locates the position of the first matched character given by user via the keyboard (*scanf()*)

Chapter 8

Structure Variable

Learning outcome

After completing this chapter, students should be able to:

1. Create a user-defined data type
2. Declare structure variable based on user-defined data type
3. Write a program to assign data to a structure variable
4. Write a program to display data of a structure variable
5. Write a program to assign data to an array of structure variable
6. Write a program to display data from an array of structure variable

Structure is a data type that can store multiple data items inside. It is also known as “*record*”. Structure’s items are designed by programmer. Each item can be any data types, depending on how a programmer designs it to meet the needs. This is the reason why structure is called “*user-defined*” data type.

In C programming, a structure is declared by the keyword “*struct*” followed by a group of its items (elements) defined within curly braces. An example of a structure is shown as follow:

```
struct std_phone {  
    int id;  
    char name[50];  
    char phone[13];  
};
```

This example is a data type used for storing student’s telephone numbers. Each record stores student’s *id* as an integer, student’s *name* as an array of characters (string), and student’s telephone number (*phone*) as an array of characters (string), respectively.

Declaring Structure Variable

Once a structure has been defined, it can be used just as other data types as `int`, `float`, `double`, etc. An example of how a structure variable is declared is illustrated as follow:

```
struct std_phone x, y;
```

From the declaration, `x` and `y` are now data objects of type `struct std_phone`. Thus, `x` and `y` contain all elements defined by the declaration. The structure of `x` and `y` is depicted in Figure 8-1.

<code>x.id</code>	2560	<code>y.id</code>	2561
<code>x.name</code>	John	<code>y.name</code>	David
<code>x.phone</code>	089-654-5487	<code>y.phone</code>	088-404-3214

Figure 8-1. Structure variable

Accessing Structure's Elements

Each element in a structure object can be accessed by referring to the structure's variable name followed by dot “.” in front of the element we want to have access to. An example of how to read data from all elements in `x` is shown below.

```
printf("ID = %d\n", x.id);
printf("Name = %s\n", x.name);
printf("Phone = %s\n", x.phone);
```

The following example shows how to assign a value to each element of `x`:

```
x.id = 2561;
strcpy(x.name, "Billy");
strcpy(x.phone, "081-345-6462");
```

As explained earlier, a string variable in C language is represented by an array of characters. In addition, since C programming does not allow assigning a value of string directly to an array of characters, as a result, function `strcpy()` is used in this case. Function `strcpy()` is a library function provided in “*string.h*” to copy a string value to an array of characters.

We can also assign a value from *stdin* to a structure's element by using *scanf()* or *puts()* just as the way we do to other built-in data types, like *int*, *float*, *double*, etc., as shown in the example below.

```
scanf("%d", &x.id);
scanf("%s", x.name);
scanf("%s", x.phone);
```

An example of how to assign a value to each element of a student's phone via *stdin* and display each element to the computer screen is given in Listing 8-1.

```
/* File: struct.c */
#include <stdio.h>

struct std_phone {
    int id;
    char name[50];
    char phone[13];
};

int main()
{
    struct std_phone david;

    printf("Enter ID: ");
    scanf("%d", &david.id);
    printf("Enter Name: ");
    scanf("%s", david.name);
    printf("Enter Phone Number: ");
    scanf("%s", david.phone);

    printf("ID: \t%d\n", david.id);
    printf("Name: \t%s\n", david.name);
    printf("Phone:\t%s\n", david.phone);
    return 0;
}
```

Listing 8-1. Assigning and printer structure variable

Array of Structure Data

Structure variable plays an important role in manipulating custom databases. Each structure variable represents a record (row) of a table. Since a table is composed of multiple records, we can then declare an array of structure variable to represent a table as we need. The following example shows how to create multiple student's phone records to form a table of student's phones.

```
struct std_phone Junior[30];
```

This will reserve 30 sets of student's phones in the memory. Each record can be accessed the same way as accessing typical array variables. Then, each element of the selected record can be accessed just as a typical structure variable. The following examples show how to read and write the *5th* record (*index of 4*) of structure array, respectively: (**Note:** array index in C language starts at 0.)

```
printf("ID: %d\n", Junior[4].id);
printf("Name: %s\n", Junior[4].name);
printf("Phone: %s\n", Junior[4].phone);
```

```
Junior[4].id = 2561;
strcpy(Junior[4].name, "Michael");
strcpy(Junior[4].phone, "801-485-3492");
```

The assignment via stdin (keyboard) is shown below:

```
scanf("%d", &Junior[4].id);
scanf("%s", Junior[4].name);
scanf("%s", Junior[4].phone);
```

Listing 8-2 demonstrates how to use an array of structure variables for creating multiple records to form a table.

```

/* File: struct_array.c */
#include <stdio.h>
#define SIZE 30
struct std_phone {
    int id;
    char name[50];
    char phone[13];
};

int main() {
    struct std_phone std[SIZE];
    int i;

    for(i=0; i<SIZE; i++) {
        printf("Enter ID [%d]: ", i);
        scanf("%d", &std[i].id);
        printf("Enter Name [%d]: ");
        scanf("%s", std[i].name);
        printf("Enter Phone Number [%d]: ");
        scanf("%s", std[i].phone);
    }
    for(i=0; i<SIZE; i++) {
        printf("ID: \t%d\n", std[i].id);
        printf("Name: \t%s\n", std[i].name);
        printf("Phone: \t%s\n", std[i].phone);
    }
    return 0;
}

```

Listing 8-2. Array of structure

typedef

Declaring a structure variable can be very inconvenient because the keyword “*struct*” has to always precede the structure name. It would be better if we can omit the this keyword. C language provides a “*typedef*” keyword, which allows us to introduce a synonym for a data type so that it can be declared in some other way. For example, we can give a new name to *int* data type by using *typedef* as follow:

```
typedef int SIZE
```

This will tell the compiler that *SIZE* can be used in place of *int* from now on. Therefore, it is legitimate to declare an integer variable by using:

```
SIZE x, y, z;
```

In this case, *x*, *y*, and *z* are variables of type *int*, which is equivalent to declaring “*int x, y, z;*”.

Using *typedef* for Structure

The keyword “*struct*” can now be omitted by using *typedef* making it more convenient for variable declaration. The following code shows how to use *typedef* to shorten the declaration of a struct data type:

```
typedef struct {  
    int x;  
    int y;  
} Point;
```

Now, the defined structure data type can be represented by the new name “*Point*”. Therefore, we can use “*Point*” to declare a new structure variable as:

```
Point p1, p2;
```

All elements (*x* and *y*) of *p1* and *p2* can then be assigned and read as the following example.

```
p1.x = 5;  
p1.y = 10;  
p2.x = 20;  
p2.y = 40;  
  
printf("p1.x = %d\n", p1.x);  
printf("p1.y = %d\n", p1.y);  
printf("p2.x = %d\n", p2.x);  
printf("p2.y = %d\n", p2.y);
```

Listing 8-3 illustrates how to use *typedef* to make a new structure data type.

```
/* File: struct_typedef.c */
#include <stdio.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    char phone[13];
} std_phone;

int main()
{
    std_phone x;

    x.id = 2018;
    strcpy(x.name, "Alex");
    strcpy(x.phone, "089-452-9583");

    printf("ID: \t%d\n", x.id);
    printf("Name: \t%s\n", x.name);
    printf("Phone:\t%s\n", x.phone);

    return 0;
}
```

Listing 8-3. Structure *typedef*

Exercise

1. Write a C program to create a structure data, which is composed of {id, name, email}. The program assigns data to the record on the source code, then prints the formatted result on the computer screen.
2. Use the table description below to create an array of structure data to store a collection of CD data. Write a C program to demonstrate how to add 3 CD records using *scanf()* and print the formatted result on the computer screen

Field	Type	Description
Album	char[40]	Store album name
Artist	char[30]	Store artist name
Year	int	Store released year
price	double	Store price of album

Chapter 9

Function

Learning outcome

After completing this chapter, students should be able to:

1. Write a function that receives a list of parameter
2. Write a function that does not receive a list parameter
3. Write a function that returns a value
4. Write a function that does not return a value
5. Write a function that receives a reference of variable
6. Write a function that receives an array of data
7. Write a program that sends a variety of data to a function
8. Write a program that receives return value from a function

Function is a set of codes (statements) that perform a specific task. Function can be called by any programs or other functions. A calling program may pass different types of arguments to the called function to accomplish a specific operation. Function can also return a value back to the calling program after it has completed its operation.

A function definition in C language consists of heading and body. Heading is simply a function definition wharease body contains statements within a pair of curly braces `{....}` to perform its specific task. The generic form of a function is given in Figure 9-1.

```
heading
{
    // Declarations (if any)
    // Statements to perform its talk
}
```

Figure 9-1. Function definition

Heading is composed of 3 main parts: *return type*, *function name*, and *list of parameter(s)*. A generic function prototype is given below.

return_type function_name(list_of_parameters);

1. *Return type* is a data type at which function returns to the calling program. It can be either built-in any data types such as *int*, *long*, *float*, *double*, *char*, any *types of pointer*, etc., or a *user-defined data type* such as *struct*, *union*, etc. If a function does not return a value, *return type* is indicated by the keyword “*void*”.
2. A *Function name* simply follows the rules defined for naming identifiers.
3. *List of parameters* receives argument value(s) (if having more than one) that caller intends to pass to function. A parameter can be any *built-in data types*, *pointers*, or a *user-defined data type* such as structure data. For multiple parameters, each will be separated by comma (“,”). However, a function that does not have a parameter can leave its parentheses empty. Some compilers may require the keyword “*void*” inside the parentheses.

Types of Function

Function is generally classified into 4 types as follows:

1. Function that does not require argument (does not contain parameter) and does not return a value.
2. Function that requires argument(s) (contains a list parameters) and does not return a value.
3. Function that does not require argument (does not contain parameter) and returns a value.
4. Function that requires argument(s) (contains a list of parameters) and returns a value.

Function with no parameter and no return

Function of this type does not contain a parameter at which the caller does not have to pass any argument value to the function. After the function completes its operation, it will not return a value back to the caller. This type of function can be used in a situation that a caller does not need to get a response back from the function. An example of a function prototype is given below.

```
void function_name();
```

The Listing 9-1 is a sample program that contains a function with no parameter and no return.

```
/* File: noarg_noret.c */
#include <stdio.h>
void print_menu(); // Function Prototype
int main() {
    print_menu(); // call to the function without argument
    return 0;
}

void print_menu() //Heading of func with no param and no return
{ // Function body
    printf("Main Menu: \n");
    printf(" 1. Add \n");
    printf(" 2. Edit \n");
    printf(" 3. Delete \n")
}
```

Listing 9-1. Function with no parameter and no return

From Listing 9-1, it is very important that a function definition has to be presented before calling it. The location of the whole function can be placed before the calling statement (before *main()*). Or it can be placed after the calling statement (as it is in the Listing). For the latter, the function prototype must be declared before a call can be made so that the caller knows how to make a call to the function. When function *main()* calls function *print_menu()*, the control of the program will be passed over to function *print_menu()*. After the function completes its operation, the control of the program is then passed back to the calling function. Then, the *main()* function resumes its operation by executing the statement after the calling statement.

Function with parameter but has no return

This type of function requires caller to pass argument(s) to the function. A function may have one or more parameters depending on how it is defined. Parameters can be any type and any order. The key point is a caller has to pass the correct argument value and order to the function. A function prototype of this type of function is given below.

```
void function_name (list_of_parameters);
```

A sample program of how to use a function that receives two parameters but has no return is demonstrated in Listing 9-2.

```
/* File: arg_noret.c */
#include <stdio.h>
void add(int , int); // Function prototype
int main()
{
    int x = 10;
    int y = 20;
    add(x, y); //calling with two arguments
    return 0;
}

void add(int a, int b) {
    int result = a + b;
    printf("%d + %d = %d\n", a, b, result);
}
```

Listing 9-2. Function with parameter(s) and has no return

From Listing 9-2, function *add()* requires two integer arguments to be passed to it. It can be noticed that we may define only the data type and omit the parameter names in the function prototype but never omit the parameter names in the function implementation. Upon a call to function, variables *x* and *y* of type *int* are passed as argument values to function *add()*, which takes these two values and assigns them to its parameters *a* and *b*, respectively. Function *add()* then adds these two parameters together and prints both of them along with the result on the computer screen. The program control is then handed over back to function *main()*.

Function with no parameter but has return

This type of function does not require caller to pass an argument value to it. When the function completes its operation, a value is passed back to the caller along with the program control. The returned value can be any data type, including pointer and structure data. A function may return a result from its operation or a value indicating whether the operation is a success or failure. A sample prototype of this type of function is given below.

```
return_type function_name();
```

A sample program of how a double value is returned from a function is shown in Listing 9-3.

```
/* File: noarg_ret.c */
#include <stdio.h>

double PI_value(); // Function Prototype

int main()
{
    double pi;
    pi = PI_value(); // calling function PI_value();
    printf("PI value = %.2f\n", pi);
    return 0;
}

double PI_value() {
    return (22.0 / 7.0);
}
```

Listing 9-3. Function with return but has no parameter

Listing 9-3 illustrates a function that returns a double value to the caller. When a call to function *PI_value()* is made, the function calculates the *PI* value and returns it back to the calling statement. Once the program control is passed by to function *main()*, *PI_value()* statement is now holding the returned value from the function. The returned value is then assigned to variable *pi* for later use.

Function with parameter and return

This type of function requires caller to pass argument value(s) to it. When the operation is done, the function returns a value back to the caller. This function type is a combination of the aforementioned function types. A sample prototype of a function with parameter(s) and returned value is given below.

return_type function_name(List_of_parameter);

An example of a function that requires 2 integer arguments and returns an integer value is shown in Listing 9-4.

```
/* File: arg_ret.c */
#include <stdio.h>
int power(int x, int y)
{
    int z = x ^ y;
    return z;
}

int main()
{
    int result;
    result = power(2, 8);
    printf("power(2, 8) = %d\n", result);
    return 0;
}
```

Listing 9-4. Function with parameter and return

From Listing 9-4, when making a call to function *power()*, two integer values must be passed to the function. In this case, two constants are sent over. After the operation of the function is done, an integer value containing the result from its calculation is returned back to the caller. The caller in function *main()* then assigns the returned value to an integer variable, *result*, for printing out later.

Note: a function prototype is needless, when the function implementation is defined before the calling statement.

Pass by value versus pass by reference

A function that receives a value of argument to its parameter is considered a *pass by value*. All the examples given so far are the type of *pass by value functions*. In this case, a copy of argument value is given to the parameter variable in function. Any changes to the parameter variable will not affect the argument variable in the calling program. A function that receives a reference of the argument variable is considered a *pass by reference*. In this case, the calling program has to send the reference (address), instead of value, of its argument to the called function. Any changes to the parameter variable in the function will also take effect on the original argument variable in the calling program. Listing 9-5 illustrates a difference of using these two types.

```

/* File: pass_value_ref.c */
#include <stdio.h>
void pass(int, int*); // Function Prototype

int main() {
    int x=10, y=20;
    printf("Before calling: x = %d -- y = %d\n", x, y);
    pass(x, &y);
    printf("After calling: x = %d -- y = %d\n", x, y);
    return 0;
}

void pass(int x, int *y) {
    x = x + 10;
    y = y + 10;
    printf("In pass(): x = %d -- y = %d\n", x, y);
}

```

Listing 9-5. Function with pass by value and reference

The result from executing this program is given below.

```

Before calling: x = 10 -- y = 20
In pass(): x = 20 -- y = 30
After calling: x = 10 -- y = 30

```

What we can see from this example is that, in function *pass()*, values of *x* and *y* are increased by 10, giving the result of *x* = 20 and *y* = 30, respectively. After the call is over, the value of *x* in function *main()* remains the same value as it is before calling. The value of *y*, on the other hand, has been increased by 10. This is because the addition of 10 to *x* in function *pass()* is made to the local variable *x* in function *pass()* itself, not to the original variable in function *main()*, whereas the addition to *y* in function *pass()* is made to the original variable in function *main()*. This is the reason why the value of *x* is unchanged in function *main()* but the value of *y* is changed.

Passing an array to a function

List of parameters may have different types and different orders. Passing a hundred of argument values, such as image pixels, to a function will be impractical for the function implementation. A better way to do so is to pass only the beginning address of an image array to function. What the function needs to know is the number of data the passed array contains. Listing 9-6 illustrates how an array is passed to a function.

```
/* File: array_pass_function.c */
#include <stdio.h>
int summation(int[ ]);
#define SIZE 10
int main() {
    int arr[SIZE] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    printf("Summation = %d\n", summation(arr));
    return 0;
}

int summation(int x[]) {
    int i, sum=0;
    for(i=0; i<SIZE; i++)
        sum += x[i];
    return sum;
}
```

Listing 9-6. Passing array to function

The result from this execution is given below.

Summation = 55

Passing a structure variable to a function

A structure variable is treated as a type of data in C programming. Thus, passing a record of a structure variable is the same as passing a typical variable. The structure argument is also passed by value. The following example in Listing 9-7 illustrates how to pass a record of a student's GPA to display in a function.

```

/* File: struct_pass_function */
#include <stdio.h>

typedef struct {
    char name[50];
    float GPA;
} std_record;

void display(std_record);

int main() {
    std_record std_gpa;
    printf("Enter name: ");
    scanf("%s", std_gpa.name);
    printf("Enter GPA: ");
    scanf("%f", &std_gpa.GPA);
    display(std_gpa);
}

void display(std_record std) {
    printf("Name: %s\n", std.name);
    printf("GPA : %.2f\n", std.GPA);
}

```

Listing 9-7. Passing a structure argument to a Function

Exercise

1. Write a program that contains 4 functions: *add()*, *subtract()*, *multiply()*, *divide()*. Each function requires two arguments of integer and returns a result of integer to the caller. Calling to all functions must be shown in the program.
2. Giving two variables of integer:
 - 2.1 write a program with a call to a function that returns the greater value .
 - 2.2 write a program with a call to a function that returns the less value .
 - 2.3 write a program with a call to a function that swaps two variables which are passed by their references.
3. Giving an array of integer storing 10 values
 - 3.1 write a program with a call to a function that returns the maximum number of the array.
 - 3.2 write a program with a call to a function that returns the minimum number of the array.
 - 3.3 write a program with a call to a function that sorts the values of the array in ascending order.
 - 3.4 write a program with a call to a function that sorts the values of the array in descending order.

Chapter 10

Command Line Argument

Learning outcome

After completing this chapter, students should be able to:

1. Write a program to receive command line arguments
2. Convert string representing number to calculatable number

C programming creates executable files (programs) that are able to run on text mode The Operating Systems such as Windows's command (cmd), Unix/Linux's Shells, or MacOS's Shell. A way to run a program on the text modes is to type the command of the program, typically the filename of that program. This type of program that allows user to interact with the program via command line is known as "*Command Line Interface*" (CLI) program as opposed to "*Graphical User Interface*" (GUI) program, which allows user to interact with the program via Button, Textbox, Checkbox, Radio Button, etc.

Since CLI programs communicate with users by the command line, which mostly uses the keyboard to send commands of data to the program to execute. In some cases, it is more convenient to pass data directly to the program without ever having the program to wait for the user's response. Command Line Argument is the way to fulfill this need. It allows users to pass data from the command line right to the program to use them for calculation or processing.

C programming allows programs to receive arguments, which are sent from command line by user, via the parameters in the *main()* function as shown in the following example:

```
#include <stdio.h>
void main(int argc, char* args[ ]) {

}
```

From the above example, function *main()* accepts two arguments of types integer and array of strings, respectively. The first argument, *argc* (argument count), stores the number of string arguments, which has been passed from the command line. The second argument, *args* (argument string), stores the array of strings, which has

been passed from the command line. An example below shows how to pass data via command line of a program named “*myapp*”:

```
./myapp one two three
```

Using the command above, *argc* stores the number of argument, in this case, *argc* = 4, while *args* stores the array of all strings as *args[]* = {“*./myapp*”, “*one*”, “*two*”, “*three*”}.

Using Command Line Argument in a Program

In fact, arguments sent from the command line are the same as typical function arguments. They can be used just as normal variables. Listing 10-1 illustrates an example of using the command line arguments.

```
/* File: cmd_line.c */
#include <stdio.h>
int main(int argc, char* args[])
{
    int i;
    for(i=0; i<argc; i++)
        printf(“args[%d] = %s\n”, i, args[i]);
    return 0;
}
```

Listing 10-1. Command line argument

From the example code in Listing 10-1, if we provide a command line “*./myapp one two three*”, then the result of running this program is displayed as follows.

```
args[0] = ./myapp
args[1] = one
args[2] = two
args[3] = three
```

In *CLI* programs, command line argument is an essential part to reduce the interaction between computer and user. For example, if a copy program (without command line argument) requires a user to enter the source file and destination file

for each copying operation, running this program 10 times means that user has to enter a pair of source and destination files 10 times to complete this operation. Most *CLI* programs support running programs in a batch file. In this case, we can pass all of source and destination files via command line arguments, list them in a batch file, and let it run until completion without having to interact with the program again. An example of a copy command is given below.

```
>copy src.txt dest.txt
```

This will pass “*src.txt*” to *args[1]* and “*dest.txt*” to *args[2]*. Then, program can make use of these two strings by opening file “*src.txt*” for reading and writing its contents to “*dest.txt*” file.

It can be noticed that the argument values passed from the command line are in the form of strings. In some cases, we may need to pass numbers to function *main()* for calculation. For example, a program called “*add*” may need to receive 2 numbers and add them up, then display the result on the screen. In this case, using command line such as “*./add 2 5*” will pass strings of “2” and “5” instead of integer values of 2 and 5. The problem is that C programming is very strict on using data types. It is not possible to add two strings together in order to get the result of type *int* or *double*. Therefore, the strings of “2” and “5” are needed to be converted to numbers, specifically of type *int*, before proceeding an addition. Fortunately, this can be done by calling function *atoi()* from *stdlib.h* to convert string data to the value of integer first. Then, addition can proceed.

Listing 10-2 demonstrates how to use library function *atoi()* to convert argument data passed from command line to integer values and then add them together.

```
/* File: asc2int.c */
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char* args[ ]) {
    int x, y, z;
    x = atoi(args[1]);
    y = atoi(args[2]);
    z = x + y;
    printf("Result of %d + %d = %d\n", x, y, z);
    return 0;
}
```

Listing 10-2. Converting string to integer

If a command line is given as “./add 2 5”, the result will be showing as

“Result of $2 + 5 = 10$ ”.

Likewise, if a number passed from command line is meant to be a floating point number, it can also be converted to a floating point number by calling function *atof()*, which will convert a string of floating point number to a number of type *double*. Listing 10-3 demonstrates how to convert two numbers representing strings of numbers of the type *double*.

```
/* File: asc2double.c */
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char* args[])
{
    double a, b, c;
    a = atof(args[1]);
    b = atof(args[2]);
    c = a + b;
    printf("Result of %.2f + %.2f = %.2f\n", a, b, c);
    return 0;
}
```

Listing 10-3. Converting string to double

If a command line is given as “./add 5.3 4.8”, the result will be showing as

“Result of $5.30 + 4.80 = 10.10$ ”.

Problem with Using Command Line Argument

Some problems caused by using command line argument include giving a wrong data type and a wrong number of arguments. In this case, the latter will be demonstrated. Supposedly, in order to run the *add* program as of the above example, the command line must provide exact 2 numbers to the program. Giving less and more will cause the program to terminate the operation without performing addition. Listing 10-4 illustrates how to protect such a problem.

```

/* File: usage.c */
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char* args[])
{
    if (argc != 3) {
        printf("ERROR: The number of argument is incorrect.\n");
        printf("\tUsage: %s <int_1> <int_2>\n", args[0]);
        printf("\t- where <int_1> and <int_2> are integer numbers.\n");
        return 0;
    }

    int x, y, z;

    x = atoi(args[1]);
    y = atoi(args[2]);
    z = x + y;

    printf("Result of %d + %d = %d\n", x, y, z);

    return 0;
}

```

Listing 10-4. Command line usage

This code example will not allow the program to proceed its addition if it does not receive the correct number of arguments, which is two values. This is to protect the program from performing unexpectedly. If giving a wrong number of arguments, the program terminates itself. However, it is a good idea to display something before leaving the program. In this case, a usage showing how to use the program is given.

Exercise

1. Write a program that accepts at least 2 integer arguments but not more than 10 from the command line. Once the program reads the arguments, converts them to integer numbers, sums them up together, and displays the summed number on the screen. (Display the usage if user enter less than 2 arguments or more than 10 arguments)
2. Write a program that accepts command line argument as usage shown below:

arith <num1> <a|s|m|d> <num2>

Where

arith is the program name

num1 and num2 are operands for arithmetic operations

a|s|m|d represents arithmetic operation selection on which

a = addition,

s = subtraction

m = multiplication

d = division

In case if user does not follow the provided format, the program will display a usage and terminate itself immediately.

Chapter 11

Stream and File

Learning outcome

After completing this chapter, students should be able to:

1. Write a program to open a file for reading
2. Write a program to open a file for writing
3. Write a program to write records of data to a file
4. Write a program to read records of data from a file

What is a file?

C language was originated from UNIX, at which it treats each device (disk, keyboard, screen, printer, etc.) in the system as a file. In order to read from or write to a file, we firstly need to open it. Then, data can be transferred from a program to the opened file, or from the file to the program. The flow of transferred data is called “*stream*”. Thus, working with a file I/O, we need to create a stream to a specific file. Essentially, after the data transfer is completed, the opened file needs to be closed.

Types of file

Generally, there are two types of files: *text* and *binary*. A text file refers to a file that contains a sequence of characters (ASCII data), while a binary file contains a series of binary data. A text file stored textual data, readable by humans, at which each line is separated by a newline character (“\n”). Text files can be opened with any text editors. In order to communicate with a text file, a textual stream is needed. A binary file, on the other hand, stores non-textual data, which are composed of a series of bytes and unreadable by humans. Some binary files can be executed by the system (.exe) while some contain data, which will be used by other programs. Some examples of binary files include picture files such as .jpg, .png, .gif, .bmp, .tif, etc., document files such as .doc, .pdf, .rtf, etc, or media files such as .mp3, .mp4, .mov, .wma, .wmv, etc. Each of these files needs designated application programs to open and manipulate it. In order to communicate with a binary file, a binary stream is needed.

File Stream

We can declare a stream of a file as a pointer to *FILE* object. The following example shows how to declare a variable of file pointer, which will be used to transfer data between program and file.

```
FILE *fptr;
```

File pointer variable *fptr* is a stream to a file that can be used to read data from or write data to the opened file. File pointer can be either text or binary stream, depending on how we set it.

Open a Stream to File

Before any transfer can proceed, a file must be opened first. A library function *fopen()*, in *stdio.h*, is used to open a text file or binary file depending on which mode we want to use. The prototype function of *fopen()* is given below:

```
FILE *fopen(const char *filename, const char *mode);
```

On successful open, *fopen()* returns a *FILE* pointer representing the stream. Otherwise, it returns a *NULL* pointer if it fails to do so. Function *fopen()* requires two arguments. The first argument is a filename, which is a string representing the file to be opened. The second argument is an open mode, which specifies properties of the stream. The details of how a file will be opened is given in Table 11-1.

Table 11-1. File open mode

Mode	Description
"r"	Open an existing text file for reading.
"w"	Create a new text file for writing at the beginning of the file. If the file already exists, its contents will be replaced.
"a"	Open an existing text file or create a new one for writing at the end of the file.
"rb"	Open existing binary file for reading.
"wb"	Create a new binary file for writing at the beginning of the file. If the file already exists, its contents will be replaced.
"ab"	Open an existing binary file or create a new one for writing at the end of file.
"r+"	Open an existing text file for reading and writing.
"w+"	Create a new text file for reading and writing at the beginning of the file. If the file already exists, its contents will be replaced.
"a+"	Open an existing text file or create a new one for reading and writing at the end of the file.
"r+b" or "rb+"	Open an existing binary file for reading and writing
"w+b" or "wb+"	Create a new binary file for reading and writing at the beginning of the file. If the file already exists, its contents will be replaced.
"a+b" or "ab+"	Open an existing binary file or create new one for reading and writing at the end of the file

Assuming that a file pointer is declared as: *FILE *fptr*, examples of how to open files are illustrated as follows:

```
fptr = fopen("test.txt", "r");
```

This statement opens a text file (*test.txt*) for reading.

```
fptr = fopen("test.bin", "rb");
```

This statement opens a binary file (*test.bin*) for reading.

```
fptr = fopen("test.txt", "w");
```

This statement opens a text file (*test.txt*) for writing. Once the file has been opened, the file pointer will be placed at the beginning of the file. As a result, all new contents will be written over the old contents. It is very important to keep this in mind. Otherwise, we will lose the old data.

```
fptr = fopen("test.txt", "a");
```

This statement opens a text file (*test.txt*) for appending. On successful open, the file pointer will be placed at the end of the file. All new contents will be written after the old contents.

Error Handling for *fopen()*

There may be some cases that a file cannot be opened. For example, if we attempt to open a file that does not exist for reading, *fopen()* will return a *NULL* pointer to indicate that a stream file cannot be established. This causes an error to the program. It is impossible to proceed with reading data. Thus, it is very important that we should check for an error occurrence on opening a file. The following example shows a simple way to check whether *fopen()* is successful.

```
fptr = fopen("input.txt", "r");
if(fptr == NULL) {
    printf("Error: cannot open file");
    return EXIT_FAILURE; // return 1;
}
```

From the example above, if the file is opened successfully, the program can proceed with reading data from the file. On the other hand, if the *fopen()* is unsuccessful, the program should terminate itself. Any operations on this file should not be done. Terminating the program is a wise choice to choose. However, it is a good idea that the program should notify the user of what is happening before terminating itself.

Close the File Stream

After file operations are over, the opened file should be closed. This will return the control over the file back to the operating system. In addition, it can prevent unexpected effects from any upcoming usages of this file by other processes. The function prototype of closing a file stream is given below.

```
int fclose(FILE *stream);
```

Function *fclose()* will return 0, if the file stream is closed successfully. Otherwise, it will return EOF. An example of closing a file stream is illustrated below.

```
int result = fclose(fp);
```

After a call of *fclose()*, the value of *fp* is no longer valid. It must not be used for any read/write operation thereafter. However, it can be used to open a file again.

Reading Data from a Text File

After a file stream has been opened successfully, it can then be used to read the contents from the file. C library provides a function to read each character from a text file as shown in function prototype below.

```
int fgetc(FILE *stream);
```

From the function prototype, given a file stream, *fgetc()* reads the next character and returns it as an unsigned char, which is casted to type of int. In case of error, the function returns *EOF* (End of file). If the read gives back a valid character, the file pointer will then move to the next character. A program can iterate the reading operation until the *EOF* is encountered. The following example shows how to read a text file and display its contents on the screen.

```
/* File: file2char.c */
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fptr;
    char c;
    fptr = fopen("test.txt", "r");
    if((fptr == NULL) {
        printf("Error: cannot open file.\n");
        return EXIT_FAILURE;
    }
    while ((c=fgetc(fptr)) != EOF)
        printf("%c", c);
    fclose(fptr);
    return EXIT_SUCCESS;
}
```

Listing 11-1. Reading character from file

Listing 11-1 opens a text file (*test.txt*) for reading. If an error occurs, the program will print out an error message and terminate itself. If the file is opened successfully, the program then reads each character and displays it on the screen until it reaches the end of file. Finally, the program closes the opened file before leaving.

Reading a Line of Characters from a File

In some cases, we might need to read multiple characters from a file. For example, we may need to read an entire line of characters. The function prototype to do so is shown below.

```
char *fgets(char *s, int size, FILE *stream);
```

Function *fgets()* accepts three parameters: *string* (pointer to character) to be read, size of character to be read, and file stream. Variable *s* is used to store characters read from the file pointer **stream*. The maximum number of characters read from the file is given by variable *size*. If *fgets()* is performed successfully, it returns the *char* pointer to the caller. On the other hand, if *fgets()* encounters *EOF*, a NULL pointer is returned with unknown contents in the buffer. An example of how to use *fgets()* is shown in Listing 11-2.

```
/* File: readline.c */
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE *fptr;
    char s[100]; //buffer of 100 characters
    fptr = fopen("test.txt", "r");
    if(fptr == NULL) {
        printf("Error: cannot open file.\n");
        return EXIT_FAILURE;
    }
    while (fgets( s, 100, fptr) != NULL)
        printf("%s\n", s);
    fclose(fptr);
    return EXIT_SUCCESS;
}
```

Listing 11-2. Reading line of characters from file

Listing 11-2 illustrates a program that reads data from a file, *test.txt*, with the maximum number of read characters of 100. If a newline character is encountered, *fgets()* stops reading data at that point and prints the entire line on screen. Upon next read, it starts reading data after the previous newline character. This gives us a characteristic of *getline* operation. The program iterates reading and printing until the EOF is reached or an error occurs.

Writing Data to a Text File

In a text file, data can be written to by two main operations: writing one character at a time and writing one line at a time. The first relies on function *fputc()* to perform its operation, while the latter uses function *fputs()* to perform its operation. The function prototype of *fputc()* is given below.

```
int fputc(int c, FILE *stream);
```

Function *fputc()* writes one character represented by (*unsigned char*) *c* to the opened file via file stream. If the operation performs successfully, it returns the character written to the file and advances the file pointer. If an error is encountered, it returns EOF. An example of how to write a character to a file is given in Listing 11-3.

```
/* File: putc2file.c */
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fptr;
    char c = 'A';
    fptr = fopen("test.txt", "r");
    if(fptr == NULL) {
        printf("Error: cannot open file.\n");
        return EXIT_FAILURE;
    }
    fputc(c, fptr);
    fclose(fptr);
    return EXIT_SUCCESS;
}
```

Listing 11-3. Writing a character to file

Listing 11-3 uses function *fputc()* to write a character 'A' to a text file, test.txt. Since the file is opened in "w" mode, character 'A' will be placed at the beginning of the file, which will write over an existing character if one exists. However, if the file is opened in append mode (*fopen()* with mode "a"), character 'A' will be placed at the end of the file.

Writing One Line at a Time

In some cases, multiple characters may need to be written to a file. For instance, an entire line of characters is to be written to a text file. This operation can be performed by using function *fputs()*, at which its function prototype is given as follows.

```
int fputs(const char *s, FILE *stream);
```

Function *fputs()* writes an array of character pointed by *char* pointer *s* to the file stream. It returns a non-negative integer upon successful operation. Otherwise, it returns *EOF* on error. An example of how to use *fputs()* to write a block of characters to a file is given in Listing 11-4.

```
/* File: block2file.c */
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fptr;
    char buf[40] = "C programming is fun.\n";
    fptr = fopen("test.txt", "r");
    if(fptr == NULL) {
        printf("Error: cannot open file.\n");
        return EXIT_FAILURE;
    }
    fputs(buf, fptr);
    fclose(fptr);
    return EXIT_SUCCESS;
}
```

Listing 11-4. Writing a block of characters to file

Listing 11-4 demonstrates how to write a string given by an array of characters (*buf*) to a text file (*test.txt*). Since the open mode is “w”, the characters (*buf*) will be written at the beginning of the file. The string “C programming is fun.” will be written over the existing characters. A newline character is placed at the end of the written string.

(Note: if the open mode is “a”, the new string will be written at the end of the file).

Writing a Block of Data to a File

A block of data may consist of textual or binary data or both. In this case, a binary file is a better choice to be used for storing such data. C programming provides function *fwrite()* to write a block of data or structure data to a file. The function prototype of *fwrite()* is given below.

```
size_t fwrite(void *ptr, size_t size, size_t nelem, FILE *stream);
```

The function accepts four arguments. The first is a *void* pointer, which can be a pointer to any types of data including *user-defined* data type such as *struct*. The second is the size of a block of data in bytes. The third is the number of blocks (elements) to be written. And the final is the file stream to be written to. Upon successful write, the function returns a value of *nelem*. If it returns a value less than *nelem*, then an error occurs. (**Note:** *size_t* is an alias name of *unsigned int*, which is already defined in the header file *stdio.h*.) Listing 11-5 illustrates an example of using *fwrite()* for writing structure data to a binary file.

```
/* File: struct2file.c */
#include <stdio.h>
typedef struct {
    char[50] name;
    float gpa;
} std_gpa;
int main() {
    std_gpa student[10];
    FILE *fptr = fopen("test.bin", "wb");
    if(fptr == NULL) {
        printf("Error: cannot open file.\n");
        return 1; // return EXIT_FAILURE;
    }
    int i;
    for(i=0; i<10; i++) {
        printf("Please enter student's name: ");
        scanf("%s", _student[i].name);
        printf("Please enter student's GPA: ");
        scanf("%f", &student[i].gpa);
    }
    fwrite(student, sizeof(std_gpa), 10, fptr);
    fclose(fptr);
    return 0;
}
```

Listing 11-5. Writing record to a file

The above sample program writes student's GPA records to a binary file. It declares objects of *std_gpa* for 10 records and receives all student's data from *stdin* (keyboard). Function *fwrite()* is given the beginning address of a student's record as the first argument, size of a record obtained from *sizeof()*, number of records to be written (10), and file stream, respectively. The program closes the file stream on completion.

Reading a Block of Data

In order to read a block of data from a file, C programming provides a function *fread()* to perform this operation. The function prototype is given below.

```
size_t fread(void *ptr, size_t size, size_t nelem, FILE *stream);
```

Function *fread()* also receives four arguments similar to its counterpart, *fwrite()*. If it returns a value less than *nelem*, either *EOF* or read error is encountered. Listing 11-6 illustrates an example of using function *fread()* for reading structure data from a binary file.

```
/* File: file2record.c */
#include <stdio.h>
typedef struct {
    char[50] name;
    float gpa;
} std_gpa;
int main() {
    std_gpa student[10];
    FILE *fptr = fopen("test.bin", "wb");
    if((fptr == NULL) {
        printf("Error: cannot open file.\n");
        return 1; // return EXIT_FAILURE;
    }
    fread(student, sizeof(std_gpa), 10, fptr);
    int i;
    for(i=0; i<10; i++) {
        printf("Student's name: %s\n", student[i].name);
        printf("Student's GPA: %.2f", student[i].gpa);
    }
    fclose(fptr);
    return 0;
}
```

Listing 11-6. Reading record from file

Manipulating Direct Access

Working with files in sequential mode can be tedious and burdensome. It is more convenient to be able to control the file pointer. The abilities to locate file pointer and move it around are necessary for direct access. The current position of a file pointer can be obtained from function *ftell()*, at which its function prototype is given below.

```
long ftell(FILE *stream);
```

Function *ftell()* requires file stream argument and returns the current position of the file pointer in terms of long integer.

For moving the file pointer to the desired position, C programming provides *fseek()* to perform this operation. The function prototype of *fseek()* is given below.

```
int fseek(FILE *stream, long offset, int mode);
```

Function *fseek()* requires 3 arguments: file stream, offset, and mode. The function moves the file pointer to the offset value according to the mode given in Table 11-2. It returns 0 if the pointer has been moved successfully. Otherwise, it returns a nonzero integer if an error occurs.

Table 11-2. File seek mode

Mode	Value	Description
SEEK_SET	0	Offset is from the beginning of the file.
SEEK_CUR	1	Offset is from the current position.
SEEK_END	2	Offset is from the end of the file.

In order to move the file pointer to the beginning of the file, C programming provides function *rewind()* to perform such operation. The function prototype of *rewind()* is given below.

```
void rewind(FILE *stream);
```

Function *rewind()* requires the specific file stream as its argument. It does not return a value. However, this function is equivalent to using *fseek(stream, 0L, SEEK_SET)*, which also moves file pointer to position 0 from the beginning of the file (SEEK_SET mode).

Exercise

1. Write a C program that opens a text file and prints its contents on computer screen
2. Write a C program that writes 3 record of *{id: int, name: char[40], phone: char[15]}* to a file
3. Write a C program that reads each record from question 2 and displays it on computer screen
4. Write a C program that reads data from a source file and write them to a destination file (Copy program)
5. Modify the program in question 4 so that it receives the source and destination files from the command line argument.

Chapter 12

Project File

Learning outcome

After completing this chapter, students should be able to:

1. Write a header file and its implementation
2. Write a program to use header file
3. Create an executable file from project file

“C language is modular” meaning that C programming is based on functions. In a large project, a source file may be full of functions. This does not make modularity more efficient. The codes become too complex, resulting in reducing the readability. Code inspection will also be very difficult to locate an issue that may cause a problem to the program. A better way to work around this problem is to break the source file into smaller multiple files. In function *main()*, we can import functions from other source files, and make a call to them as needed. Thus, any errors that might arise in the project will be isolated making it easier to locate the problem. In addition, this will also improve the modularity of the program.

C programming allows us to keep functions in separate files. These file can be imported for use in our program using directive *#include*. Furthermore, the interfaces of functions (function prototypes) and their implementations can be stored separately. Traditionally, the function prototypes and variable declarations are kept in a header file (*.h file*), while the function implementations are kept in a source file (*.c file*).

The following example illustrates how a C project can be created. The project structure consists of 3 files: a header file containing function prototypes shown in Listing 12-1, a source file containing the implementations of all functions defined in the header file shown in Listing 12-2, and a main source file containing function *main()* to launch the execution and all main control of the program shown in Listing 12-3.

```
/* File: arith.h */  
// Header file containing function prototypes  
int add(int, int);  
int subtract(int, int);
```

Listing 12-1. Header file (arith.h)

```

/* File: arith.c */
//Source file containing implementation of
// all function defined in Header file
int add(int x, int y)
{
    int result = x + y;
    return result;
}
int subtract(int x, int y)
{
    int result = x - y;
    return result;
}

```

Listing 12-2. Source file (arith.c)

```

/* File: main.c */
// Source file containing function main();
#include <stdio.h>
#include "arith.h"
int main()
{
    int a = 10, b = 5, c = 0;
    c = add(a, b);
    printf("%d + %d = %d\n", a, b, c);
    c = subtract(a, b);
    printf("%d - %d = %d\n", a, b, c);
    return 0;
}

```

Listing 12-3. Source file (main.c)

From the above example, it can be seen that the source file that implements all functions in the header file must have the same name as the header file (*arith.c* and *arith.h*). But what is very interesting is that we include the header file (*arith.h*) without specifying its source file in the main file. Since the main file needs to know only how to call the functions, there is no need to know their implementations.

In order to make an executable file, all the source files must be compiled and linked together. In this case, we have two source files, *arith.c* and *main.c*. Both must be compiled to create object files. Then, both object files must be linked together so that all the functions can be located. The executable file from the above project can be created by using *gcc* as follows:

1. Compile all source files to generate object files:

```
$gcc -c arith.c main.c
```

This *-c* option tells *gcc* to generate *arith.o* and *main.o*

2. Link all object files together to generate an executable file

```
$gcc arith.o main.o -o main
```

This command lets *gcc* link the object files together. The *-o* option tells *gcc* to generate an executable file named *main*. If the *-o* and the target file name are omitted, the executable file will automatically be named *a.out* on Linux and *a.exe* on Windows.

(Note: the commands will be different on different compilers.)

In a large project, there may be so many header and source files to keep track with. It is possible that a header file may be included more than once. Consequently, the compiler will compile its contents more than once. As a result, the compiler gives out an error because of the duplicate contents. A typical approach to deal with this problem is to enclose the entire contents in the header file with a conditional preprocessor directive as shown in Listing 12-4.

```
/* File: arith.h (new arith.h) */
// Header file with conditional directive
#ifndef ARITH_H
#define ARITH_H
int add(int, int);
int subtract(int, int);
#endif
```

Listing 12-4. Modified Header file

The preprocessor directive `#ifndef` checks whether a macro name (`ARITH_H`) has not been defined. On the first read, `ARITH_H` has not yet been defined. Therefore, it is defined here and all subsequent contents are read. Later, if this header file is included again, it first checks for the Macro name, `ARITH_H`, which has just been defined previously. As a result, all contents within `#ifndef` block will be skipped.

Exercise

1. Add more functions to the header file, then write the function implementation and make function calls on new added functions
2. A header file contains function prototypes as followings:

```
int RectangleArea(int, int); // (width, height)
float TriangleArea(int, int); // (base, height)
double CircleArea(double); // (radius)
```

Write the following files:

Header file: `area.h`, containing the above function prototypes (interfaces)
 Source file: `area.c`, implementing all functions in header file
 Source file `testArea.c.`, containing `main()`;

3. Give the commands to make a executable file for the project in question 2

Bibliography

Neill Graham, “**Learning C**”, International Edition, McGraw-Hill, Inc. 1992, ISBN: 0-07-112628-7

Steve Oualline, “**Practical C Programming**”, 3rd Edition, O'Reilly Media, Inc., August 1997, ISBN: 1565923065, ISBN-13: 978-1565923065

Tony Zhang, “**Sams Teach Yourself C in 24 Hours**”, 2nd Edition, Sams Publishing, February 2000, ISBN-10: 067231861X, ISBN-13: 2900672318619

Bradley L. Jones and Peter Aitken, “**Teach Yourself C in 21 Days**”, Sams Publishing, October 2002, ISBN-10: 0672324482, ISBN-13: 978-0672324482

Tutorialspoint, “**Learn C Programming**”, <http://www.tutorialspoint.com>